```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.P01.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S01_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot       Mock  →   →     T001④
    ///      Boot            UI T001b  Boot      =
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.P01.Boot
{
    /// &lt;summary&gt;
```

```
/// Boot     T001
/// ①Logger→②  →③Config→④Save→⑤ILocale→⑥Ads.Preload→⑦         [BOOT] n/7
///  ⑦        Play  =SceneManager.LoadSceneAsync Editor dry-run=EditorSceneManager.OpenScene
/// X006      SY_MOCK
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &amp;= Run(1, "Logger", LoggerStep);
        ok &amp;= Run(2, "Time", TimeStep);
        ok &amp;= Run(3, "Config", ConfigStep);
        ok &amp;= Run(4, "Save", SaveStep);
        ok &amp;= Run(5, "ILocale", LocaleStep);
        ok &amp;= Run(6, "Ads.Preload", AdsStep);
        ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; //              JS sink platform T001c
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
    }
```

```
        static bool ConfigStep()
        {
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock=    X009   manifest
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont); //         +
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.P01.Boot
{
    /// &lt;summary&gt;
    ///    Mock    SY_MOCK    X006     Mock X002a   AppID touristappid
    /// T001c  WX/TT     Channel   Editor      Boot
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; //
            ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
            ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
            ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
            ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
            // T021c-ii: launch-query seam (null = organic open; the platform
            // suite swaps this registration when its channel face lands T001c).
            ServiceLocator.Register&lt;ILaunchInfo&gt;(new MockLaunchInfo());
```

```
        // T011: the shared AdCapPolicy is registered BEFORE the ads service so the
        // revive pre-check (HasCategoryBudget) and ShowRewarded consumption read the
        // same §  category counters (P02 boot paradigm; p01_revive 6/day). The
        // ad service is the shared-package mock (red lines + lifecycle events) -
        // the old local MockAdsService ignored caps entirely.
        var adCap = new AdCapPolicy();
        ServiceLocator.Register(adCap);
        ServiceLocator.Register&lt;IAdsService&gt;(new Sy.Platform.MockAdsService(adCap));
        Logger.Info("BOOT", "mock services registered (SY_MOCK)");
    }
}
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK X006                  TCB serverTime→HTTPS Date →   T001b   &lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;X006             T001b v1→v2     &lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// &lt;summary&gt;T035 i18n slice 1: resolves through the shipped cfg_i18n
    /// table (P01Configs.I18n); a missing key falls back to the raw key
    /// and warns (the T001b contract, now wired - AboutPanel keeps its own
    /// zero-service resolver for the headless gate face). Format args
    /// apply only when the caller actually passes them.&lt;/summary&gt;
    public string GetText(string key, params string[] args)
```

```
        {
            var table = Sy.P01.Newbie.P01Configs.I18n;
            if (table != null &amp;amp;&amp;amp; table.entries != null)
            {
                foreach (var e in table.entries)
                {
                    if (e != null &amp;amp;&amp;amp; e.key == key &amp;amp;&amp;amp; e.value != key)
                    {
                        return args != null &amp;amp;&amp;amp; args.Length &amp;gt; 0 ? string.Format(e.value, args) : e.value;
                    }
                }
            }
            Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
            return key;
        }
        public FontSet GetFontSet()
            =&amp;gt; new FontSet("P01FontCJK", "P01FontCJK", "P01FontCJK"); // T035: bundled CJK subset (device-safe chain)
    }
    public sealed class MockLaunchInfo : ILaunchInfo
    {
        /// &amp;lt;summary&amp;gt;SY_MOCK Editor/      query=             T001c  suite   &amp;lt;/summary&amp;gt;
        public LaunchQuery GetLaunchQuery() =&amp;gt; null;
    }
}
// MainUI.cs
using UnityEngine;
using UnityEngine.SceneManagement;
namespace Sy.P01.Newbie
{
    public static class NewbieSceneIds
    {
        public const string Main = "S01_Main";
        public const string Battle = "S01_Battle";
    }
    /// &amp;lt;summary&amp;gt;S01          →S01_Battle M5      ——
    /// X027 art wiring: swaps the scene's solid Bg block for the generated scene
    /// painting and shows the hero showcase when the art keys resolve.
    /// T017 slice 3: hosts the DayLauncher (day/night loop day beat) on this scene
    /// root - AddComponent pattern keeps the scene YAML untouched.
    /// T018 slice 3: hosts the TalentLauncher AFTER the DayLauncher so the talent
    /// entry button renders above the day panel (later sibling order).&amp;lt;/summary&amp;gt;
    public sealed class MainUI : MonoBehaviour
    {
        public void StartNight()
        {
            SceneManager.LoadScene(NewbieSceneIds.Battle);
        }
        void Awake()
        {
            Application.targetFrameRate = 30;
```

```
            Sy.Core.MainArtInjector.Apply(transform, "P01_scene_01", "P01_unit_01");
            if (GetComponent&lt;DayLauncher&gt;() == null)
                gameObject.AddComponent&lt;DayLauncher&gt;();
            if (GetComponent&lt;TalentLauncher&gt;() == null)
                gameObject.AddComponent&lt;TalentLauncher&gt;();
            // T045: compliance about entry - self-hosting programmatic panel
            // (P06 T024 pattern, zero scene-YAML drift), after the talent
            // host so the disclosure overlay renders above the day panel.
            if (GetComponent&lt;AboutPanel&gt;() == null)
                gameObject.AddComponent&lt;AboutPanel&gt;();
            // T033 slice A: first-launch privacy gate - hosted LAST so the
            // privacy overlay renders above every menu face on first launch
            // (deny = cannot-enter hint state, agree = local persisted bit;
            // the X279 self-hosting pattern, zero scene-YAML drift).
            if (GetComponent&lt;PrivacyPanel&gt;() == null)
                gameObject.AddComponent&lt;PrivacyPanel&gt;();
            // T035: scene-serialized Text objects still reference the builtin
            // font (zero CJK glyphs -&gt; tofu on device); sweep them to the
            // bundled subset at boot. Scene YAML untouched (zero drift), the
            // swap is idempotent and play-mode only.
            foreach (var txt in UnityEngine.Object.FindObjectsByType&lt;UnityEngine.UI.Text&gt;(
                         UnityEngine.FindObjectsInactive.Include, UnityEngine.FindObjectsSortMode.None))
            {
                if (txt.font != null &amp;&amp; txt.font.name == "LegacyRuntime.ttf")
                    txt.font = Sy.P01.UiFont.Resolve();
            }
        }
    }
}
// AssemblyInfo.cs
using System.Runtime.CompilerServices;
// T011: editor gates (Sy.P01.Editor) stage lethal moments through
// BattleSim.ForceHeroHpForTests; gameplay assemblies never see the internals.
[assembly: InternalsVisibleTo("Sy.P01.Editor")]
// UiFont.cs
using System;
using Sy.Core;
using UnityEngine;
namespace Sy.P01
{
    /// &lt;summary&gt;
    /// T035 font chain: resolve the UI font through the locale FontSet instead
    /// of hardcoding the engine built-in font.
    /// Why: LegacyRuntime.ttf carries no CJK glyphs and WebGL/WeChat builds
    /// have no OS font fallback, so every Chinese label would render blank
    /// on device (editor stays green via OS fallback - a false-comfort mask,
    /// the U184 portfolio-wide disease). The bundled subset asset P01FontCJK.ttf
    /// (Noto Sans SC OFL, subset to this game's own character census, 232KB,
    /// FontGate missing=0) covers the whole UI vocabulary; X027 law: missing
    /// asset falls back to the built-in font + one warning, never breaks a
```

```
    /// build or gate. P02 FateGate U184 pattern replicated verbatim.
    /// &lt;/summary&gt;
    public static class UiFont
    {
        const string Fallback = "LegacyRuntime.ttf";
        static bool _warned;
        public static Font Resolve()
        {
            string name = null;
            try
            {
                if (ServiceLocator.TryGet&lt;ILocaleService&gt;(out var locale))
                    name = locale.GetFontSet().NormalFont;
            }
            catch (Exception) { name = null; }
            if (!string.IsNullOrEmpty(name) &amp;&amp; name != Fallback)
            {
                var font = Resources.Load&lt;Font&gt;("Fonts/" + name);
                if (font != null) return font;
                if (!_warned)
                {
                    Sy.Core.Logger.Warn("UIFONT", "font asset missing, fallback LegacyRuntime: " + name);
                    _warned = true;
                }
            }
            return Resources.GetBuiltinResource&lt;Font&gt;(Fallback);
        }
    }
}
// UiText.cs
using System;
using Sy.Core;
namespace Sy.P01
{
    /// &lt;summary&gt;
    /// T035 i18n slice 2: the shared UI text resolver (the UiFont kin). The
    /// canonical ILocaleService.GetText seam runs first (play mode - the
    /// registered service owns localization when the platform suite swaps
    /// one in at T001c); the zero-service cfg_i18n table resolver runs
    /// second so edit-mode gates drive real renders without Boot
    /// (FriendBoardCheck F7 builds this panel headless). One semantics
    /// everywhere (T001b contract): a missing key falls back to the raw key
    /// (the service face warns), format args apply only when the caller
    /// passes them, so brace-bearing values never Format-crash on zero-arg
    /// calls. The migration queue (BattleView -&gt; DayView -&gt; ...) keys its
    /// labels through this seam batch by batch; the shipped table copy is
    /// the only place CJK lives.
    /// &lt;/summary&gt;
    public static class UiText
    {
```

```
        public static string Get(string key, params string[] args)
        {
            bool hasArgs = args != null &amp;&amp; args.Length &gt; 0;
            try
            {
                if (ServiceLocator.TryGet&lt;ILocaleService&gt;(out var locale))
                    return locale.GetText(key, args);
            }
            catch (Exception) { /* fall through to the zero-service face */ }
            // Edit-mode/gate face: resolve the shipped table directly (the
            // AboutPanel.TryText kin; value == key counts as missing).
            string value = key;
            var table = Newbie.P01Configs.I18n;
            if (table != null &amp;&amp; table.entries != null)
            {
                foreach (var e in table.entries)
                {
                    if (e != null &amp;&amp; e.key == key &amp;&amp; e.value != key) { value = e.value; break; }
                }
            }
            return hasArgs ? string.Format(value, args) : value;
        }
    }
}
// AboutPanel.cs
using System;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.P01.Newbie
{
    /// &lt;summary&gt;
    /// T045 compliance surface - the about/disclosure face on the main menu
    /// (the P06 T024 AboutPanel pattern). Every disclosure line is composed
    /// FROM the live law tables (cfg_rune rerollFace pool, cfg_ad_point
    /// global cap) so the published numbers can never drift from the
    /// enforced ones - the same construction guarantee the P02 odds panel
    /// uses. Programmatic UGUI, zero scene-YAML drift; the strings resolve
    /// through the shipped cfg_i18n table (raw-key fallback). The composer
    /// layer is pure static the headless gate asserts without rendering.
    /// &lt;/summary&gt;
    public sealed class AboutPanel : MonoBehaviour
    {
        public const string KeyAboutButton = "about_button";
        public const string KeyPanelTitle = "about_title";
        public const string KeyAigcLine = "about_aigc_line";
        public const string KeyOddsLine = "about_odds_line";
        public const string KeyAdCapLine = "about_adcap_line";
        public const string KeyAgeLine = "about_age_line";
        public const string KeyCloseLabel = "about_close";
        /// Draw width of the bell_reroll re-draw (3 of the six-rune pool,
```

```
/// BattleSim.DrawRerollFaces law) - kept beside the composer so the
/// disclosure and the engine read one constant.
public const int RerollDrawWidth = 3;
static readonly Color LanternAmber = new Color(0.85f, 0.62f, 0.25f);
static readonly Color PaperWhite = new Color(0.95f, 0.91f, 0.84f);
static readonly Color InkBrown = new Color(0.24f, 0.20f, 0.18f);
static readonly Color NightDim = new Color(0.09f, 0.07f, 0.13f, 0.92f);
// ──── composer layer (pure static, gate-facing) ────
/// Strings resolver over the shipped cfg_i18n table (the ILocaleService
/// semantics: a missing key falls back to the raw key and warns).
public static bool TryText(string key, out string value)
{
    value = key;
    var table = P01Configs.I18n;
    if (table == null || table.entries == null) return false;
    foreach (var e in table.entries)
    {
        if (e != null &amp;&amp; e.key == key) { value = e.value; return e.value != key; }
    }
    return false;
}
public static string Text(string key)
{
    string value;
    return TryText(key, out value) ? value : key;
}
/// AIGC explicit disclosure: art and audio assets in this game are
/// AI-assisted (platform AI-generated-content labeling rule; the
/// implicit platform-side declaration stays an admin item).
public static string AigcLine()
{
    return Text(KeyAigcLine);
}
/// Random-draw disclosure. The published numbers are formatted
/// straight out of the live rerollFace pool (cfg_rune) and the draw
/// width law, so the shown face and the enforced law are the same
/// object - a hot-tuned pool re-renders the disclosure with zero
/// extra wiring. Uniform draw, no rarity tiers, no paid draws.
public static string OddsLine()
{
    int pool = NewbieConfig.RerollFaces.Length;
    int draw = Math.Min(RerollDrawWidth, pool);
    int pct = (int)Math.Round(draw * 100.0 / Math.Max(1, pool));
    return string.Format(Text(KeyOddsLine), pool, draw, pct);
}
/// Ad-cap disclosure: the global daily ceiling formatted out of the
/// live ad-point law (voluntary views only - no forced ad).
public static string AdCapLine()
{
    var law = P01Configs.AdPoint;
```

```
    int cap = law != null ? law.globalCapDaily : 0;
    return string.Format(Text(KeyAdCapLine), cap);
}
/// Age-rating disclosure (micro-horror folk theme: 12+).
public static string AgeLine()
{
    return Text(KeyAgeLine);
}
// ——— view layer ———
RectTransform _overlay;
void Start()
{
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    if (canvas == null) return;   // headless gate: nothing to host on
    Initialize(canvas);
}
void Initialize(Canvas canvas)
{
    // persistent entry button, bottom-right of the main menu
    var entryGo = new GameObject("AboutEntry", typeof(RectTransform), typeof(Image), typeof(Button));
    entryGo.transform.SetParent(canvas.transform, false);
    var entry = (RectTransform)entryGo.transform;
    entry.anchorMin = entry.anchorMax = new Vector2(1f, 0f);
    entry.anchoredPosition = new Vector2(-140f, 96f);
    entry.sizeDelta = new Vector2(150f, 64f);
    var entryImg = entryGo.GetComponent&lt;Image&gt;();
    entryImg.sprite = MakeSquareSpriteShared();
    entryImg.color = LanternAmber;
    var entryTxt = MakeText(entry, Text(KeyAboutButton), 34, Vector2.zero,
        TextAnchor.MiddleCenter, 150f, 64f);
    entryTxt.color = PaperWhite;
    entryTxt.raycastTarget = false;
    entryGo.GetComponent&lt;Button&gt;().onClick.AddListener(Open);
    // disclosure overlay (hidden until the entry opens it)
    _overlay = new GameObject("AboutOverlay", typeof(RectTransform), typeof(Image)).GetComponent&lt;RectTransfor
    _overlay.transform.SetParent(canvas.transform, false);
    _overlay.anchorMin = Vector2.zero;
    _overlay.anchorMax = Vector2.one;
    _overlay.offsetMin = _overlay.offsetMax = Vector2.zero;
    var dim = _overlay.GetComponent&lt;Image&gt;();
    dim.sprite = MakeSquareSpriteShared();
    dim.color = NightDim;
    var paper = MakeCard(_overlay, "AboutPaper", Vector2.zero, new Vector2(940, 1180),
        new Color(PaperWhite.r, PaperWhite.g, PaperWhite.b, 0.97f));
    MakeText(paper, Text(KeyPanelTitle), 64, new Vector2(0, 460),
        TextAnchor.MiddleCenter, 900, 110).color = InkBrown;
    MakeText(paper, AigcLine(), 36, new Vector2(0, 250),
        TextAnchor.MiddleCenter, 820, 120).color = InkBrown;
    MakeText(paper, OddsLine(), 34, new Vector2(0, 60),
        TextAnchor.MiddleCenter, 820, 190).color = InkBrown;
```

```
        MakeText(paper, AdCapLine(), 34, new Vector2(0, -130),
            TextAnchor.MiddleCenter, 820, 110).color = InkBrown;
        MakeText(paper, AgeLine(), 34, new Vector2(0, -250),
            TextAnchor.MiddleCenter, 820, 110).color = InkBrown;
        MakeButton(paper, Text(KeyCloseLabel), new Vector2(0, -440),
            new Vector2(420, 96), Close);
        _overlay.gameObject.SetActive(false);
    }
    public void Open()
    {
        if (_overlay != null) _overlay.gameObject.SetActive(true);
    }
    void Close()
    {
        if (_overlay != null) _overlay.gameObject.SetActive(false);
    }
    // ——— tools (programmatic UGUI, zero scene YAML) ———
    static RectTransform MakeCard(RectTransform parent, string name, Vector2 pos, Vector2 size, Color color)
    {
        var go = new GameObject(name, typeof(RectTransform), typeof(Image));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = size;
        var img = go.GetComponent&lt;Image&gt;();
        img.sprite = MakeSquareSpriteShared();
        img.color = color;
        return rt;
    }
    static Text MakeText(RectTransform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float
    {
        var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(w, h);
        var t = go.GetComponent&lt;Text&gt;();
        t.font = UiFont.Resolve();               // T035: FontSet-driven CJK chain
        t.text = text;
        t.fontSize = size;
        t.alignment = anchor;
        t.color = InkBrown;
        return t;
    }
    static void MakeButton(RectTransform parent, string label, Vector2 pos, Vector2 size, Action onClick)
    {
        var go = new GameObject("ActionButton", typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(parent, false);
```

```
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = pos;
            rt.sizeDelta = size;
            var img = go.GetComponent&lt;Image&gt;();
            img.sprite = MakeSquareSpriteShared();
            img.color = LanternAmber;
            var txt = MakeText(rt, label, 40, Vector2.zero, TextAnchor.MiddleCenter, size.x, size.y);
            txt.color = PaperWhite;
            txt.raycastTarget = false;
            go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; onClick());
        }
        static Sprite _sharedSquare;
        static Sprite MakeSquareSpriteShared()
        {
            if (_sharedSquare == null)
            {
                var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
                var px = tex.GetPixels32();
                for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
                tex.SetPixels32(px);
                tex.Apply(false, false);
                _sharedSquare = Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
            }
            return _sharedSquare;
        }
    }
}
// AdPointFunnel.cs
using System.Collections.Generic;
using Sy.Platform;
namespace Sy.P01.Newbie
{
    /// &lt;summary&gt;
    /// T020 slice 2 - the cfg_ad_point consumer face: the P02 three-piece pattern
    /// (HasCategoryBudget pre-check -&gt; category-gated consume -&gt; evt_ad funnel
    /// telemetry) as pure functions over the shared AdCapPolicy, so the slice-3
    /// point UI wiring and the AdFunnelCheck gate stay thin call sites. Per-run
    /// caps stay sim-side (BattleSim counters, the revive T011 precedent); this
    /// face owns the day pools, the gain decay ladder, the P1-11 revive gate and
    /// the interstitial rules. Zero server (red line 12) - policy and table are
    /// both local; a missing table row reads as "no offer" (fail-loud callers
    /// surface the missing asset through P01Configs.Load's error log).
    /// &lt;/summary&gt;
    public static class AdPointFunnel
    {
        /// &lt;summary&gt;Runtime ad point id: p01_-isolated namespace (revive T011
        /// precedent) - the ads service and evt_ad telemetry never see a bare id
        /// that could collide with the P02/P06/G namespaces.&lt;/summary&gt;
        public static string AdPointId(string pointId) =&gt; "p01_" + pointId;
```

```
/// &lt;summary&gt;Table row for a cfg point id (null = unknown id).&lt;/summary&gt;
public static P01Configs.AdPointRow Row(string pointId)
{
    var table = P01Configs.AdPoint;
    if (table?.points == null) return null;
    foreach (var p in table.points)
        if (p.pointId == pointId) return p;
    return null;
}
/// &lt;summary&gt;Section-six pre-offer gate (P02 paradigm): true when the
/// point's p01_ category still has day budget. The global 12 cap is
/// enforced at consume; per-run caps are sim-side and owned by the host.&lt;/summary&gt;
public static bool PrecheckBudget(string pointId, string dayKey, AdCapPolicy policy)
{
    var row = Row(pointId);
    if (row == null || row.type != "rewarded") return false;
    return policy.HasCategoryBudget(dayKey, AdCategories.CategoryOfPoint(AdPointId(pointId)));
}
/// &lt;summary&gt;T020 slice 3b-3i: prep-face offer decision, pure so AdFunnelCheck
/// K9 can pin the red line without a scene. Run 1 IS the tutorial run
/// (redline11 sibling): its pick face never shows an in-run ad offer, even
/// with a full budget; every later run rides the normal budget precheck.&lt;/summary&gt;
public static bool PrepOfferLive(string pointId, int runCount, string dayKey, AdCapPolicy policy)
{
    if (runCount == 1) return false;
    return PrecheckBudget(pointId, dayKey, policy);
}
/// &lt;summary&gt;Category-gated consume (P02 paradigm): day pool first, then
/// the global 12 hard cap; both counters advance together. False = the
/// host tracks the funnel "blocked" state and never retries the ad.&lt;/summary&gt;
public static bool TryConsume(string pointId, string dayKey, AdCapPolicy policy)
{
    var row = Row(pointId);
    if (row == null || row.type != "rewarded") return false;
    return policy.TryConsumeRewarded(dayKey, AdCategories.CategoryOfPoint(AdPointId(pointId)));
}
/// &lt;summary&gt;Gain decay face (task book L15): rescue-class points keep full
/// value on every watch (decay null, constant full); gain-class points walk
/// the ladder by same-day watch index (watch 1 = 1.0, 2 = 0.9, 3 = 0.8,
/// 4+ = 0.7). watchIdx is 1-based; a missing table reads as full value.&lt;/summary&gt;
public static float GainMul(string pointId, int watchIdx)
{
    var row = Row(pointId);
    var ladder = P01Configs.AdPoint?.gainDecayLadder;
    if (row == null || ladder == null || ladder.Length == 0) return 1f;
    if (string.IsNullOrEmpty(row.decay)) return 1f;
    int idx = watchIdx &lt;= 1 ? 0
        : watchIdx &gt; ladder.Length ? ladder.Length - 1
        : watchIdx - 1;
    return ladder[idx];
```

```
        }
        /// &lt;summary&gt;Revive second-watch gate (red-team P1-11): the second revive
        /// offer only arms late (wave &gt;= 12 OR run duration &gt;= 240s) so a strong
        /// early run cannot farm the 6/day rescue pool. First watch is ungated.&lt;/summary&gt;
        public static bool ReviveGateOk(int wave, float durationSec, bool secondWatch)
        {
            if (!secondWatch) return true;
            var gate = Row("revive")?.gate;
            if (gate == null) return true;
            return wave &gt;= gate.waveGte || durationSec &gt;= gate.orDurationSecGte;
        }
        /// &lt;summary&gt;Interstitial face: the first-5-runs exemption (L121 verbatim)
        /// gates before the shared policy's 180s cooldown consume.&lt;/summary&gt;
        public static bool InterstitialAllowed(int runCount, string dayKey, float nowSec, AdCapPolicy policy)
        {
            var row = Row("interstitial");
            if (row == null) return false;
            int exempt = row.gate?.firstRunsExempt ?? 0;
            if (runCount &lt; exempt) return false;
            return policy.TryConsumeInterstitial(dayKey, nowSec);
        }
        /// &lt;summary&gt;evt_ad funnel telemetry (P01 TrackRevive field precedent:
        /// state/pointId/wave/reason - the emitted pointId is the p01_-isolated
        /// runtime id). Slice-3 hosts pass their boot-registered (or launcher
        /// fallback) TelemetryService; states = offer/accept/ad_fail/decline/
        /// blocked/complete per the P02 funnel vocabulary.&lt;/summary&gt;
        public static void Track(Sy.Telemetry.TelemetryService telemetry, string state,
            string pointId, int wave, string reason)
        {
            if (telemetry == null) return;
            var fields = new Dictionary&lt;string, string&gt;
            {
                { "state", state },
                { "pointId", AdPointId(pointId) },
            };
            if (wave &gt;= 0) fields["wave"] = wave.ToString();
            if (reason != null) fields["reason"] = reason;
            telemetry.Track(Sy.Telemetry.EventIds.Ad, fields);
        }
    }
}
// BattleLauncher.cs
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.EventSystems;
namespace Sy.P01.Newbie
{
    /// &lt;summary&gt;
    /// S01_Battle       BattleSim+TutorialFlow+BattleView   Tick+
    ///    Sy.Telemetry Editor =Debug sink T001e        +TCB
```

```
/// HUD                    6
/// &lt;/summary&gt;
public sealed class BattleLauncher : MonoBehaviour
{
    BattleSim _sim;
    TutorialFlow _tutorial;
    BattleView _view;
    Sy.Telemetry.TelemetryService _telemetry;
    Sy.Social.ShareCard _reportSpec;      // T021b: the settle-face report card spec
    Sy.Social.ComposeResult _reportCard;  // T021b: the composed report card (png path)
    Sy.Platform.AdCapPolicy _adCap;   // T011: shared boot policy (or session fallback)
    bool _revivePairShown;            // T011: mirrors the offer pair visibility
    bool _pullbackPlayed;             // T012: camera beat is intro-only (Fighting re-entries from talisman/revive m
    bool _dragging;
    Vector2 _dragOrigin;
    int _lastWave = -1, _lastCoins = -1, _lastKills = -1, _lastHp = -1, _lastCountdown = -1;
    const string RevivePointId = "p01_revive";   // T011: §  point 1 (p01_-isolated namespace)
    // T020 slice 3b: settle_double site state - the bonus face anchors the
    // wave-reward settle face (60+40 per cfg_item); the funnel takes the bare
    // point id and derives the p01_ runtime id.
    const string SettleDoublePointId = "settle_double";   // T020: §  point 2 (settle coins x2)
    const string InterstitialPointId = "interstitial";   // T020: §  interstitial row (post_run)
    const string StartTalentPointId = "start_talent";   // T020: §  point 3 (prep pair, uiStyle prep_panel)
    const string WeaponTrialPointId = "weapon_trial";   // T020: §  point 4 (prep pair, uiStyle prep_panel)
    const string DyingShieldPointId = "dying_shield";    // T020: §  point 3 (death-face shield)
    const string SoulLampPointId = "soul_lamp";          // T020: §  point 6 (boss-wave burn window)
    const string BellRerollPointId = "bell_reroll";       // T020: §  point 7 (smart-trigger re-draw)
    int _settleBonusFace;
    bool _settleDoubleLive;
    bool _prepTalentLive;   // T020 slice 3b-3i: prep-pair liveness (per pause face, settled on watch)
    bool _prepTrialLive;
    bool _dyingShieldLive;      // T020 slice 3b-2: shield offer holds the death run open
    bool _nightRunClosed;       // T020 slice 3b-2: one close per run (the shield-open nav face may tap after a clos
    bool _lampLive;             // T020 slice 3b-3ii: boss-wave lamp offer liveness (settled on watch)
    bool _rerollLive;           // T020 slice 3b-3iii: bell_reroll offer liveness (settled on watch)
    void Awake()
    {
        Application.targetFrameRate = 30;
        var canvas = FindObjectOfType&lt;Canvas&gt;();
        // T019 slice 2: the local save face hydrates BEFORE the sim consumes the
        // run-count / codex faces (red line 12 - local file is the authority; a
        // missing file = honest fresh D1 + run 1).
        NewbieSession.HydrateFromSave();
        _sim = new BattleSim(seed: 20260916);
        _sim.SetRunCount(NewbieSession.RunCount);
        // T023 slice 2: rotate the battlefield for this night run (cycle_by_run
        // over the dayIndex-unlocked roster, cfg_map) and stamp the formal-spawn
        // modifiers onto the sim (single numeric truth, the SetRunCount precedent).
        MapService.SelectCurrent(NewbieSession.RunCount, NewbieSession.DayState.DayIndex);
        _sim.SetMapModifiers(MapService.CurrentSpeedMul,
```

```
    MapService.PaperEffigyTemplateId, MapService.CurrentPaperHpPct);
_sim.SetCodexCollected(NewbieSession.TalentState.CodexCollected.Count);
_view = gameObject.AddComponent&lt;BattleView&gt;();
_view.Initialize(canvas, _sim);
_telemetry = new Sy.Telemetry.TelemetryService();
_telemetry.AddSink(new TelemetryDebugSink());
// T021c-ii: point-card receive face - a share-card cold start carries
// the friend's build on the boot query's bd= token; it becomes THIS
// run's opening build (apply-by-replace, one-shot). Organic opens
// stay byte-identical: no launch face (headless) reads as absent.
BuildImport.CaptureFromPlatform();
var importedBuild = BuildImport.TryTakeBuild();
if (importedBuild != null &amp;&amp; _sim.ApplyImportedBuild(importedBuild))
    _view.SetHint(UiText.Get("battle_import_hint"));
_tutorial = new TutorialFlow(
    _sim,
    hint =&gt;
    {
        Debug.Log("[HINT] " + hint);
        _view.SetHint(hint);
    },
    step =&gt; _telemetry.Track(Sy.Telemetry.EventIds.NewbieStep,
        new Dictionary&lt;string, string&gt; { { "step", step.ToString() } }));
_view.TalismanPickRequested += OnTalismanPickRequested;
_sim.StateChanged += OnStateChanged;
_sim.EnemyKilled += OnEnemyKilled;          //          §4.1
_sim.TalismanPicked += OnTalismanPicked;    //       V1.1
_sim.ChainEvolvedDone += OnChainEvolved;    //     →
// T012 slice 2: soul-bottle presentation beats (§0 T+4-10, sim-clock driven
// so the View   and the Redline11Check probes share one timeline).
_sim.BottleLidHit += OnBottleLidHit;
_sim.BottleHeadPoke += OnBottleHeadPoke;
// T016 slice 3: boss enrage + capture-bottle show (shareable moment 3) -
// the rune-short window shows the "so close" hint, the executed capture
// plays the boss-into-bottle beat.
_sim.BossEnraged += OnBossEnraged;
_sim.BossCaptureMarked += OnBossCaptureMarked;
_sim.BossCaptured += OnBossCaptured;
// T011 revive traffic (p01_revive point, §  point 1, P02 rescue paradigm):
// the boot-registered AdCapPolicy is the same instance the suite's
// ShowRewarded consumes from, so the pre-check and the consumption stay
// consistent; without boot the fallback policy scopes the cap to this
// session (headless baselines never reach here - they drive the sim only).
if (!Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.AdCapPolicy&gt;(out _adCap))
    _adCap = new Sy.Platform.AdCapPolicy();
_sim.ReviveOffered += OnReviveOffered;
_sim.ReviveOfferExpired += OnReviveOfferExpired;
_sim.Revived += OnRevived;
_sim.DyingShielded += OnDyingShielded;   // T020 slice 3b-2: the shield resume beat
_sim.LampOffered += OnLampOffered;       // T020 slice 3b-3ii: the boss-wave lamp arm
```

```
        _sim.RerollOffered += OnRerollOffered;   // T020 slice 3b-3iii: the re-offer bell arm
        _view.ReviveAcceptRequested += OnReviveAcceptRequested;
        _view.ReviveDeclineRequested += OnReviveDeclineRequested;
        _view.SettleDoubleRequested += OnSettleDoubleRequested;   // T020 slice 3b
        _view.ShareRequested += OnShareRequested;                 // T021b: report-card share
        _view.FriendBoardRequested += OnFriendBoardRequested;    // T022-iii: friend board
        _view.PrepTalentRequested += OnPrepTalentRequested;   // T020 slice 3b-3i
        _view.PrepTrialRequested += OnPrepTrialRequested;
        _view.DyingShieldRequested += OnDyingShieldRequested;     // T020 slice 3b-2
        _view.LampAcceptRequested += OnLampAcceptRequested;       // T020 slice 3b-3ii
        _view.RerollAcceptRequested += OnRerollAcceptRequested;    // T020 slice 3b-3iii
        _view.DeathNavBattleRequested += () =&gt; LeaveDeathFace(NewbieSceneIds.Battle);
        _view.DeathNavHomeRequested += () =&gt; LeaveDeathFace(NewbieSceneIds.Main);
    }
    void Update()
    {
        TickJoystickInput();
        _sim.Tick(Time.deltaTime);
        _view.SyncView(_sim);
        if (_sim.SoulBurstPending)
        {
            _view.PlaySoulBurst();
            _sim.ConsumeSoulBurst();
        }
        // T011: countdown label mirrors the sim's 3s window (int-change only).
        if (_sim.ReviveOfferActive &amp;amp;&amp;amp; _revivePairShown)
        {
            int left = Mathf.CeilToInt(_sim.ReviveCountdownSeconds);
            if (left != _lastCountdown)
            {
                _lastCountdown = left;
                _view.SetReviveCountdown(left);
            }
        }
        if (_sim.Wave != _lastWave) { _lastWave = _sim.Wave; _view.SetWave(_lastWave); }
        if (_sim.Coins != _lastCoins) { _lastCoins = _sim.Coins; _view.SetCoins(_lastCoins); }
        if (_sim.Kills != _lastKills) { _lastKills = _sim.Kills; _view.SetKills(_lastKills); }
        if ((int)_sim.HeroHp != _lastHp) { _lastHp = (int)_sim.HeroHp; _view.SetHp(_lastHp); }
    }
    /// &lt;summary&gt;      §10.2.2             =       UI      &lt;/summary&gt;
    void TickJoystickInput()
    {
        if (Input.GetMouseButtonDown(0)
            &amp;amp;&amp;amp; (EventSystem.current == null || !EventSystem.current.IsPointerOverGameObject()))
        {
            _dragging = true;
            _dragOrigin = Input.mousePosition;
        }
        if (Input.GetMouseButtonUp(0))
        {
```

```
            _dragging = false;
            _sim.SetMoveInput(0f, 0f);
            _view.HideJoystick();
        }
        if (!_dragging) return;
        var delta = (Vector2)Input.mousePosition - _dragOrigin;
        if (delta.magnitude &gt; 12f)
        {
            _sim.SetMoveInput(delta.x, delta.y);
            _view.ShowJoystick(_dragOrigin, Input.mousePosition);
        }
        else
        {
            _sim.SetMoveInput(0f, 0f);
            _view.HideJoystick();
        }
    }
    void OnTalismanPickRequested(int index)
    {
        _sim.PickTalisman(index);
    }
    void OnTalismanPicked(int index)
    {
        _view.SetChain(_sim.ChainDisplay);
    }
    void OnChainEvolved()
    {
        _view.SetChain(_sim.ChainDisplay);
    }
    void OnEnemyKilled(BattleSim.GhostInst g)
    {
        _view.PlaySoulAbsorb(g.X, g.Y);
        // T012 slice 2: the FIRST kill is the §0 triple-feedback beat - count+1
        // (SetKills), soul fire (absorb above) and the break sfx all land together.
        if (_sim.Kills == 1) _view.PlayBreakSfx();
    }
    // T012 slice 2: §0 bottle beats - the lid "dong" and the head-poke wow moment.
    void OnBottleLidHit(float simTime) =&gt; _view.PlayBottleLidBeat();
    void OnBottleHeadPoke(float simTime) =&gt; _view.PlayBottleHeadPoke();
    // T016 slice 3: enrage red-light beat + the capture-window faces. The capture
    // itself executes sim-side on the same tick the gate opens, so a "ready" boss
    // never shows the miss hint - BossCaptured fires right after the mark.
    void OnBossEnraged(BattleSim.GhostInst g) =&gt; _view.PlayBossEnrageShow();
    void OnBossCaptureMarked(BattleSim.GhostInst g)
    {
        if (!_sim.IsBossCaptureReady(g))
            _view.ShowCaptureMissHint(g.CaptureRunes - _sim.TalismanPicks);
    }
    void OnBossCaptured(BattleSim.GhostInst g)
    {
```

```
        _view.PlayBossCaptureShow(g.X, g.Y);
        // T019 slice 3: first-encounter capture registration - the capture
        // GUARANTEES the codex entry (05 s9 suspender next to the week-chain
        // belt); the live count re-stamps the battle's collection bonus on the
        // same tick, the collect beat lands right after the capture show, and
        // the guaranteed collect persists immediately (crash-safe collection).
        if (NewbieSession.RegisterCodexCapture(g.TemplateId))
        {
            _sim.SetCodexCollected(NewbieSession.TalentState.CodexCollected.Count);
            NewbieSession.WriteToSave();
            string label = LauncherTelemetry.CodexSpeciesLabel(g.TemplateId);
            _view.PlayCodexCollectBeat(label);
            LauncherTelemetry.TrackCodexCollect(_telemetry, label,
                NewbieSession.TalentState.CodexCollected.Count);
        }
    }
    void OnStateChanged(BattleSim.BattleState state)
    {
        switch (state)
        {
            case BattleSim.BattleState.Fighting:
                _view.HideIntro();
                // T012 slice 2: §0 T+1.5-3.0s camera beat rides the intro-to-street
                // handover (zoomed on the bearer, eases out as the swarm closes in).
                // Fighting re-entries (talisman resume / revive) never replay it.
                if (!_pullbackPlayed)
                {
                    _pullbackPlayed = true;
                    _view.PlayCameraPullback();
                }
                break;
            case BattleSim.BattleState.TalismanPause:
                _view.ShowTalismanPanel();
                MaybeShowPrepOffers();   // T020 slice 3b-3i: the pick face doubles as the prep face
                break;
            case BattleSim.BattleState.Won:
                // T017 slice 3: the won night closes the day loop - bottle ghosts
                // (=Kills headcount since X209) + coins feed tomorrow's day layer.
                // T018 slice 3: the same deposit completes the chain verb - the
                // auto-claimed codex grant lands on the settle as its own beat.
                // T019 slice 2: the settle grants the pending wave rewards as coins
                // (cfg_item waveRewardFace) on top of the in-run counter, then the
                // night's run closes and the whole session persists.
                int settleCoins = _sim.Coins + _sim.PendingRewardCoins;
                int runIdx = NewbieSession.RunCount;   // T020: pre-increment 1-based run index (interstitial exempt
                NewbieSession.DepositNight(_sim.BottleGhosts, settleCoins);
                CloseNightRun();
                _view.ShowSettle(_sim.Kills, settleCoins);
                if (NewbieSession.LastChainDay &gt; 0)
                    _view.SetSettleCodex(UiText.Get("battle_settle_codex_fmt",
```

```
                NewbieSession.LastChainDay.ToString(), NewbieSession.LastChainEnemyId,
                NewbieSession.LastChainCoins.ToString()));
            // T020 slice 3b: settle_double offer (§  point 2, P02 T015-A2
            // paradigm) - the bonus face = the wave-reward settle face
            // (60+40); no budget, no dead-end button.
            _settleBonusFace = _sim.PendingRewardCoins;
            _settleDoubleLive = false;
            if (_settleBonusFace &gt; 0
                &amp;&amp; AdPointFunnel.PrecheckBudget(SettleDoublePointId,
                    Sy.Platform.AdCapPolicy.TodayKey(), _adCap))
            {
                _settleDoubleLive = true;
                // The offer label shows what THIS watch grants - the decay
                // ladder rides the same-day watch index (pre-consume +1).
                int watchIdxNext = _adCap.UsedCategoryToday(Sy.Platform.AdCapPolicy.TodayKey(),
                    Sy.Platform.AdCategories.CategoryOfPoint(AdPointFunnel.AdPointId(SettleDoublePointId))) + 1;
                _view.ShowSettleDoubleOffer(Mathf.RoundToInt(
                    _settleBonusFace * AdPointFunnel.GainMul(SettleDoublePointId, watchIdxNext)));
                AdPointFunnel.Track(_telemetry, "offer", SettleDoublePointId, _sim.Wave, "settle_coins");
            }
            // T020 slice 3b: post-run interstitial (uiStyle post_run; the
            // death face rework lands with dying_shield in the next slice).
            ComposeReportCard();   // T021b: report card + share offer (Won = the one legal ReadPixels stall poi
            MaybeFirePostRunInterstitial(runIdx);
            break;
        case BattleSim.BattleState.Lost:
            // T024-ii: the BattleLost-face wave-fail emitter (04 Schema
            // section 5) - one honest beat per real lost run, through the
            // shared seam the marathon gate also drives.
            LauncherTelemetry.TrackWaveFail(_telemetry, MapService.CurrentMapId, _sim.Wave);
            // T011: the death settle. The sim evaluates the revive offer BEFORE
            // this event, so a live offer means the pair is (or was budget-
            // checked to be) up; no pair shown = final form right away.
            _view.ShowDeathSettle(_sim.Kills, _sim.Coins, _sim.Wave);
            if (!_revivePairShown)
            {
                // T020 slice 3b-2: terminal death face (no revive pair) - the
                // dying_shield offer is the weaker second-chance tier (1/run
                // sim-side + p01_shield 3/day pre-check, no dead-end button).
                // While the offer holds, the run stays open (an accept resumes
                // the battle) and the nav pair rides alongside so leaving is
                // still one tap - the nav route closes the run on exit.
                _dyingShieldLive = false;
                if (!_sim.DyingShieldUsed
                    &amp;&amp; AdPointFunnel.PrecheckBudget(DyingShieldPointId,
                        Sy.Platform.AdCapPolicy.TodayKey(), _adCap))
                {
                    var shieldRow = AdPointFunnel.Row(DyingShieldPointId);
                    if (shieldRow != null)
                    {
```

```
                        _dyingShieldLive = true;
                        _view.ShowDyingShieldOffer(shieldRow.invulnSec, shieldRow.healPct);
                        AdPointFunnel.Track(_telemetry, "offer", DyingShieldPointId, _sim.Wave, "death_face");
                    }
                }
                _view.ShowDeathNav();
                if (!_dyingShieldLive)
                {
                    // No shield face to answer: close + the post-run interstitial
                    // death face (the Won face landed with slice 3b-1).
                    int closeRunIdx = NewbieSession.RunCount;   // pre-close 1st-of-day index (site exemption fa
                    CloseNightRun();
                    MaybeFirePostRunInterstitial(closeRunIdx);
                }
            }
            break;
    }
}
// ——— T011 revive traffic (p01_revive, §  funnel point 1) ———
void OnReviveOffered(int wave, int reviveIndex)
{
    // §  pre-offer gate: with no p01_revive budget (6/day) left the offer is
    // silently skipped instead of showing a dead-end button (P02 paradigm).
    string day = Sy.Platform.AdCapPolicy.TodayKey();
    if (!_adCap.HasCategoryBudget(day, Sy.Platform.AdCategories.CategoryOfPoint(RevivePointId))) return;
    _revivePairShown = true;
    _lastCountdown = -1;
    _view.ShowReviveOffer();
    TrackRevive("offer", wave, reviveIndex, null);
}
async void OnReviveAcceptRequested()
{
    if (!_sim.ReviveOfferActive) return;
    // The tap commits the offer: freeze the 3s window while the ad runs so a
    // real-channel ad (which can outlast 3s) can never expire the offer the
    // player already accepted.
    _sim.HoldReviveOfferForAd();
    _view.HideReviveOffer();
    if (Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IAdsService&gt;(out var ads))
    {
        // Channel ad path (mock in Editor): the suite consumes the shared
        // policy inside ShowRewarded; awaiting keeps the real-channel JS
        // callback flow free of main-thread blocking.
        var result = await ads.ShowRewarded(RevivePointId);
        if (!result.Completed)
        {
            _sim.DeclineRevive();
            _revivePairShown = false;
            ShowDeathNavAndCloseRun();
            TrackRevive("ad_fail", _sim.Wave, _sim.ReviveCount + 1, result.Reason);
```

```
            return;
        }
    }
    else
    {
        // Degraded session (no boot): consume locally against the fallback cap.
        if (!_adCap.TryConsumeRewarded(Sy.Platform.AdCapPolicy.TodayKey(),
                Sy.Platform.AdCategories.CategoryOfPoint(RevivePointId)))
        {
            _sim.DeclineRevive();
            _revivePairShown = false;
            ShowDeathNavAndCloseRun();
            TrackRevive("ad_fail", _sim.Wave, _sim.ReviveCount + 1, "daily_cap");
            return;
        }
    }
    if (_sim.AcceptRevive())
        TrackRevive("complete", _sim.Wave, _sim.ReviveCount, null);
}
void OnReviveDeclineRequested()
{
    if (!_sim.ReviveOfferActive) return;
    _sim.DeclineRevive();
    _revivePairShown = false;
    ShowDeathNavAndCloseRun();
    TrackRevive("decline", _sim.Wave, _sim.ReviveCount + 1, null);
}
void OnReviveOfferExpired()
{
    _revivePairShown = false;
    _view.HideReviveOffer();
    ShowDeathNavAndCloseRun();
    TrackRevive("decline", _sim.Wave, _sim.ReviveCount + 1, "timeout");
}
// T019 slice 2: a terminal death closes the night run - the difficulty
// schedule counts attempts, not wins - and persists the session.
// T020 slice 3b-2: every death-path close fires the post-run interstitial
// on the death face (uiStyle post_run; the shield-held run fires it at
// the nav exit instead).
void ShowDeathNavAndCloseRun()
{
    _view.ShowDeathNav();
    int runIdx = NewbieSession.RunCount;   // pre-close index (site exemption face)
    CloseNightRun();
    MaybeFirePostRunInterstitial(runIdx);
}
void CloseNightRun()
{
    // T020 slice 3b-2: idempotent one-close guard - with the shield offer
    // holding the run open, a nav tap can arrive after an ad_fail close;
```

```
    // a second increment would double-count the attempt face.
    if (_nightRunClosed) return;
    _nightRunClosed = true;
    NewbieSession.RunCount++;
    NewbieSession.WriteToSave();
}
// T020 slice 3b-2: the death nav routes through the host so an exit from
// the shield-held death face still closes the run (run-count + interstitial
// death face) before the scene swap; an already-closed run (revive declined,
// shield ad_fail) rides straight into the load.
void LeaveDeathFace(string sceneId)
{
    if (!_nightRunClosed)
    {
        int runIdx = NewbieSession.RunCount;
        CloseNightRun();
        MaybeFirePostRunInterstitial(runIdx);
    }
    UnityEngine.SceneManagement.SceneManager.LoadScene(sceneId);
}
// ──── T020 slice 3b-2: dying_shield traffic (§  point 3) ────
async void OnDyingShieldRequested()
{
    if (!_dyingShieldLive) return;
    _dyingShieldLive = false;
    _view.HideDyingShieldOffer();
    string day = Sy.Platform.AdCapPolicy.TodayKey();
    if (Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IAdsService&gt;(out var ads))
    {
        // Channel ad path (mock in Editor): the suite consumes the shared
        // policy inside ShowRewarded - the same instance _adCap holds in a
        // booted session (settle_double 3b-1 paradigm).
        var result = await ads.ShowRewarded(AdPointFunnel.AdPointId(DyingShieldPointId));
        if (!result.Completed)
        {
            AdPointFunnel.Track(_telemetry, "ad_fail", DyingShieldPointId, _sim.Wave, result.Reason);
            ShowDeathNavAndCloseRun();
            return;
        }
    }
    else if (!AdPointFunnel.TryConsume(DyingShieldPointId, day, _adCap))
    {
        // Degraded session (no boot): consume locally against the fallback
        // cap; a false here is the daily-cap blocked face (never retried).
        AdPointFunnel.Track(_telemetry, "blocked", DyingShieldPointId, _sim.Wave, "daily_cap");
        ShowDeathNavAndCloseRun();
        return;
    }
    // Nav-exit race guard: while the ad was in flight the death nav could
    // close the run and start the scene swap - never resume a closed run
```

```
    // (the destroyed-view continuation would crash on the resume beat).
    if (_nightRunClosed || _view == null) return;
    var row = AdPointFunnel.Row(DyingShieldPointId);
    if (!_sim.AcceptDyingShield(row != null ? row.invulnSec : 0f, row != null ? row.healPct : 0f))
    {
        // Sim-side refusal (shield already used / a live revive pair) - the
        // honest face is the closed nav, never a stuck open panel.
        AdPointFunnel.Track(_telemetry, "decline", DyingShieldPointId, _sim.Wave, "sim_refused");
        ShowDeathNavAndCloseRun();
        return;
    }
    AdPointFunnel.Track(_telemetry, "complete", DyingShieldPointId, _sim.Wave, null);
}
/// &lt;summary&gt;The shield resume beat (sim DyingShielded event): the death settle
/// folds away and the run resumes in place - the dodge window is the point.&lt;/summary&gt;
void OnDyingShielded(int wave)
{
    _view.HideDeathSettle();
    _view.PlayDyingShieldFlash();
    _view.SetHint(UiText.Get("battle_shield_hint"));
}
// ──── T020 slice 3b-3ii: soul_lamp traffic (§  point 6) ────
// The sim arms the offer at a director boss wave (1/run cap sim-side);
// this host owns the p01_lamp day pre-check (3/day), the decay-adjusted
// label and the evt_ad funnel. The grant lights the burn window with
// the ladder-scaled magnitudes (label = grant, the honest-face precedent).
// A wave that clears while a real-channel ad is in flight refuses the
// accept (sim_refused, logged) - the untouched 1/run cap re-arms at the
// next boss wave, so no watch is silently swallowed.
void OnLampOffered(int wave)
{
    if (!_sim.LampOfferActive) return;
    string day = Sy.Platform.AdCapPolicy.TodayKey();
    if (!AdPointFunnel.PrecheckBudget(SoulLampPointId, day, _adCap)) return;
    var row = AdPointFunnel.Row(SoulLampPointId);
    if (row == null) return;
    _lampLive = true;
    int watchIdxNext = _adCap.UsedCategoryToday(day,
        Sy.Platform.AdCategories.CategoryOfPoint(AdPointFunnel.AdPointId(SoulLampPointId))) + 1;
    _view.ShowLampOffer(LampFace(row, AdPointFunnel.GainMul(SoulLampPointId, watchIdxNext)));
    AdPointFunnel.Track(_telemetry, "offer", SoulLampPointId, wave, "boss_wave");
}
/// &lt;summary&gt;Decay-adjusted label face: what THIS watch grants (the same
/// ladder the grant applies - label and grant must never disagree).&lt;/summary&gt;
static string LampFace(P01Configs.AdPointRow row, float mul)
{
    int dmgPct = Mathf.RoundToInt(row.dmgPct * mul * 100f);
    float soul = row.soulMul * mul;
    string soulFace = Mathf.Approximately(soul, Mathf.Round(soul))
        ? Mathf.RoundToInt(soul).ToString()
```

```
            : soul.ToString("F1");
        return UiText.Get("battle_lamp_offer_fmt", dmgPct.ToString(),
            Mathf.RoundToInt(row.durationSec).ToString(), soulFace);
    }
    async void OnLampAcceptRequested()
    {
        if (!_lampLive) return;
        _lampLive = false;
        _view.HideLampOffer();
        string day = Sy.Platform.AdCapPolicy.TodayKey();
        if (Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IAdsService&gt;(out var ads))
        {
            // Channel ad path (mock in Editor): the suite consumes the shared
            // policy inside ShowRewarded - the same instance _adCap holds.
            var result = await ads.ShowRewarded(AdPointFunnel.AdPointId(SoulLampPointId));
            if (!result.Completed)
            {
                _sim.DeclineLamp();
                AdPointFunnel.Track(_telemetry, "ad_fail", SoulLampPointId, _sim.Wave, result.Reason);
                return;
            }
        }
        else if (!AdPointFunnel.TryConsume(SoulLampPointId, day, _adCap))
        {
            // Degraded session (no boot): the daily-cap blocked face, never retried.
            _sim.DeclineLamp();
            AdPointFunnel.Track(_telemetry, "blocked", SoulLampPointId, _sim.Wave, "daily_cap");
            return;
        }
        // Gain ladder rides the same-day watch index (post-consume read-back,
        // the settle_double precedent); the burn magnitudes scale by it.
        int watchIdx = _adCap.UsedCategoryToday(day,
            Sy.Platform.AdCategories.CategoryOfPoint(AdPointFunnel.AdPointId(SoulLampPointId)));
        var row = AdPointFunnel.Row(SoulLampPointId);
        if (row == null) return;
        float mul = AdPointFunnel.GainMul(SoulLampPointId, watchIdx);
        if (_sim.AcceptLamp(row.dmgPct * mul, row.durationSec, row.soulMul * mul))
        {
            _view.PlayLampFlash();
            _view.SetHint(UiText.Get("battle_lamp_burn_hint_fmt",
                Mathf.RoundToInt(row.dmgPct * mul * 100f).ToString(),
                Mathf.RoundToInt(row.durationSec).ToString()));
            AdPointFunnel.Track(_telemetry, "complete", SoulLampPointId, _sim.Wave, null);
        }
        else
        {
            // Sim-side refusal (the wave ended while the ad was in flight) -
            // the honest face is no burn window, never a phantom grant.
            AdPointFunnel.Track(_telemetry, "decline", SoulLampPointId, _sim.Wave, "sim_refused");
        }
```

```
}
// ——— T020 slice 3b-3iii: bell_reroll traffic (§  point 7) ———
// The sim arms the bell on a re-offer where the fixed trio duplicates the
// held build (smart trigger + 1/run cap sim-side); this host owns the
// p01_reroll day pre-check (3/day) and the evt_ad funnel. The draw pool
// is cfg_rune rerollFace (single numeric authority) - the gain ladder
// stays bookkeeping-only for this point (the row has no magnitudes to
// scale, faces are never decayed). A pick that lands while a real-channel
// ad is in flight refuses the accept (sim_refused, logged) - the untouched
// cap re-arms at the next qualifying pause, so no watch is silently
// swallowed.
void OnRerollOffered(int pauseIndex)
{
    if (!_sim.RerollOfferActive) return;
    string day = Sy.Platform.AdCapPolicy.TodayKey();
    if (!AdPointFunnel.PrecheckBudget(BellRerollPointId, day, _adCap)) return;
    if (AdPointFunnel.Row(BellRerollPointId) == null) return;
    _rerollLive = true;
    _view.ShowRerollOffer();
    AdPointFunnel.Track(_telemetry, "offer", BellRerollPointId, pauseIndex, "talisman_reoffer");
}
async void OnRerollAcceptRequested()
{
    if (!_rerollLive) return;
    _rerollLive = false;
    _view.HideRerollOffer();
    string day = Sy.Platform.AdCapPolicy.TodayKey();
    if (Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IAdsService&gt;(out var ads))
    {
        // Channel ad path (mock in Editor): the suite consumes the shared
        // policy inside ShowRewarded - the same instance _adCap holds.
        var result = await ads.ShowRewarded(AdPointFunnel.AdPointId(BellRerollPointId));
        if (!result.Completed)
        {
            _sim.DeclineReroll();
            AdPointFunnel.Track(_telemetry, "ad_fail", BellRerollPointId, _sim.Wave, result.Reason);
            return;
        }
    }
    else if (!AdPointFunnel.TryConsume(BellRerollPointId, day, _adCap))
    {
        // Degraded session (no boot): the daily-cap blocked face, never retried.
        _sim.DeclineReroll();
        AdPointFunnel.Track(_telemetry, "blocked", BellRerollPointId, _sim.Wave, "daily_cap");
        return;
    }
    if (_sim.AcceptReroll())
    {
        _view.ApplyRerollFaces(_sim.OfferedFacesNow);
        _view.PlayRerollFlash();
```

```
            _view.SetHint(UiText.Get("battle_reroll_hint"));
            AdPointFunnel.Track(_telemetry, "complete", BellRerollPointId, _sim.Wave, null);
        }
        else
        {
            // Sim-side refusal (the pause closed while the ad was in flight) -
            // the honest face is no re-draw, never a phantom draw.
            AdPointFunnel.Track(_telemetry, "decline", BellRerollPointId, _sim.Wave, "sim_refused");
        }
    }
    void OnRevived(int wave)
    {
        _revivePairShown = false;
        _view.HideDeathSettle();
        _view.PlayReviveFlash();
        _view.SetHint(UiText.Get("battle_revive_hint"));
    }
    void TrackRevive(string state, int wave, int reviveIndex, string reason)
    {
        var fields = new Dictionary&lt;string, string&gt;
        {
            { "state", state },
            { "pointId", RevivePointId },
            { "wave", wave.ToString() },
            { "reviveIdx", reviveIndex.ToString() },
        };
        if (reason != null) fields["reason"] = reason;
        _telemetry.Track(Sy.Telemetry.EventIds.Ad, fields);
    }
    // ——— T020 slice 3b: settle_double traffic (§  point 2) ———
    async void OnSettleDoubleRequested()
    {
        if (!_settleDoubleLive) return;
        _settleDoubleLive = false;
        _view.HideSettleDoubleOffer();
        string day = Sy.Platform.AdCapPolicy.TodayKey();
        if (Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IAdsService&gt;(out var ads))
        {
            // Channel ad path (mock in Editor): the suite consumes the shared
            // policy inside ShowRewarded - the same instance _adCap holds in a
            // booted session, so the post-consume read-back stays truthful.
            var result = await ads.ShowRewarded(AdPointFunnel.AdPointId(SettleDoublePointId));
            if (!result.Completed)
            {
                AdPointFunnel.Track(_telemetry, "ad_fail", SettleDoublePointId, _sim.Wave, result.Reason);
                return;
            }
        }
        else if (!AdPointFunnel.TryConsume(SettleDoublePointId, day, _adCap))
        {
```

```
            // Degraded session (no boot): consume locally against the fallback
            // cap; a false here is the daily-cap blocked face (never retried).
            AdPointFunnel.Track(_telemetry, "blocked", SettleDoublePointId, _sim.Wave, "daily_cap");
            return;
        }
        // Gain ladder rides the same-day watch index - UsedCategoryToday
        // already counts this watch after the consume (watch 1 = 1.0 full).
        int watchIdx = _adCap.UsedCategoryToday(day,
            Sy.Platform.AdCategories.CategoryOfPoint(AdPointFunnel.AdPointId(SettleDoublePointId)));
        int bonus = Mathf.RoundToInt(_settleBonusFace * AdPointFunnel.GainMul(SettleDoublePointId, watchIdx));
        if (bonus &gt; 0)
        {
            // The accepted bonus re-deposits the settle face - base + bonus IS
            // the x2 (the ad reward mints nothing else; wallet = economy truth).
            NewbieSession.WalletCoins += bonus;
            NewbieSession.WriteToSave();   // crash-safe (T019 pattern)
            _view.PlaySettleDoubleFlash(bonus);
        }
        AdPointFunnel.Track(_telemetry, "complete", SettleDoublePointId, _sim.Wave, null);
    }
    // ──── T020 slice 3b-3i: prep pair traffic (§  points 3+4) ────
    // The three-pick face doubles as the level-prep face (X228 probe: the
    // pitch book has no standalone prep page). Run 1 IS the tutorial run -
    // PrepOfferLive pins the zero-offer red line pure for AdFunnelCheck K9.
    // Each watch grants the cfg coins face x the decay ladder into the
    // shared wallet (settle_double precedent); nothing spends wallet coins
    // mid-run, so the weapon_trial "next run" semantic holds without a
    // second economy truth.
    void MaybeShowPrepOffers()
    {
        string day = Sy.Platform.AdCapPolicy.TodayKey();
        _prepTalentLive = AdPointFunnel.PrepOfferLive(StartTalentPointId, NewbieSession.RunCount, day, _adCap);
        _prepTrialLive = AdPointFunnel.PrepOfferLive(WeaponTrialPointId, NewbieSession.RunCount, day, _adCap);
        int talentCoins = 0, trialCoins = 0;
        if (_prepTalentLive)
        {
            talentCoins = PrepCoinsFace(StartTalentPointId, day);
            AdPointFunnel.Track(_telemetry, "offer", StartTalentPointId, _sim.Wave, "prep_talent");
        }
        if (_prepTrialLive)
        {
            trialCoins = PrepCoinsFace(WeaponTrialPointId, day);
            AdPointFunnel.Track(_telemetry, "offer", WeaponTrialPointId, _sim.Wave, "prep_trial");
        }
        _view.ShowPrepOffers(talentCoins, trialCoins);
    }
    /// &lt;summary&gt;Offer label face: what THIS watch grants - the cfg coins
    /// magnitude x the decay ladder on the pre-consume watch index.&lt;/summary&gt;
    int PrepCoinsFace(string pointId, string day)
    {
```

```
    var row = AdPointFunnel.Row(pointId);
    if (row == null) return 0;
    int watchIdxNext = _adCap.UsedCategoryToday(day,
        Sy.Platform.AdCategories.CategoryOfPoint(AdPointFunnel.AdPointId(pointId))) + 1;
    return Mathf.RoundToInt(row.coins * AdPointFunnel.GainMul(pointId, watchIdxNext));
}
async void OnPrepTalentRequested()
{
    if (!_prepTalentLive) return;
    _prepTalentLive = false;
    _view.HidePrepTalentOffer();
    await GrantPrepWatch(StartTalentPointId);
}
async void OnPrepTrialRequested()
{
    if (!_prepTrialLive) return;
    _prepTrialLive = false;
    _view.HidePrepTrialOffer();
    await GrantPrepWatch(WeaponTrialPointId);
}
async System.Threading.Tasks.Task GrantPrepWatch(string pointId)
{
    string day = Sy.Platform.AdCapPolicy.TodayKey();
    if (Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IAdsService&gt;(out var ads))
    {
        // Channel ad path (mock in Editor): the suite consumes the shared
        // policy inside ShowRewarded - the same instance _adCap holds.
        var result = await ads.ShowRewarded(AdPointFunnel.AdPointId(pointId));
        if (!result.Completed)
        {
            AdPointFunnel.Track(_telemetry, "ad_fail", pointId, _sim.Wave, result.Reason);
            return;
        }
    }
    else if (!AdPointFunnel.TryConsume(pointId, day, _adCap))
    {
        // Degraded session (no boot): the daily-cap blocked face, never retried.
        AdPointFunnel.Track(_telemetry, "blocked", pointId, _sim.Wave, "daily_cap");
        return;
    }
    // Gain ladder rides the same-day watch index - UsedCategoryToday
    // already counts this watch after the consume (watch 1 = 1.0 full).
    int watchIdx = _adCap.UsedCategoryToday(day,
        Sy.Platform.AdCategories.CategoryOfPoint(AdPointFunnel.AdPointId(pointId)));
    var row = AdPointFunnel.Row(pointId);
    int bonus = row != null ? Mathf.RoundToInt(row.coins * AdPointFunnel.GainMul(pointId, watchIdx)) : 0;
    if (bonus &gt; 0)
    {
        NewbieSession.WalletCoins += bonus;
        NewbieSession.WriteToSave();   // crash-safe (T019 pattern)
```

```
        _view.PlayPrepCoinsFlash(bonus);
    }
    AdPointFunnel.Track(_telemetry, "complete", pointId, _sim.Wave, null);
}
// ——— T020 slice 3b: post-run interstitial (§  interstitial row) ———
/// &lt;summary&gt;uiStyle post_run (the Won settle face since 3b-1; the death face
/// since 3b-2 fires at the terminal close - the shield-held run defers it
/// to the nav exit). The site owns the first-5-runs exemption (L121) so the
/// suite's cooldown consume never fires for exempt runs; the suite consumes
/// the 180s cooldown + cap internally (Mock/Wx/Douyin parity - never a
/// second consume here), the degraded session rides the funnel face.&lt;/summary&gt;
void MaybeFirePostRunInterstitial(int runIdx)
{
    int exempt = AdPointFunnel.Row(InterstitialPointId)?.gate?.firstRunsExempt ?? 0;
    if (exempt &gt; 0 &amp;&amp; runIdx &lt; exempt) return;   //  5
    if (Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IAdsService&gt;(out var ads))
        ads.ShowInterstitial(AdPointFunnel.AdPointId(InterstitialPointId));
    else
        AdPointFunnel.InterstitialAllowed(runIdx, Sy.Platform.AdCapPolicy.TodayKey(),
            Time.realtimeSinceStartup, _adCap);
}
// ——— T021b: settle-face report-card share (red line 11 share moment) ———
/// &lt;summary&gt;Won is the one legal ReadPixels stall point (P02 law): compose
/// the T021a report card from the closing run's stats; only a clean card
/// raises the share button (no card = no dead-end button, P02 paradigm).
/// Teaching-run zero-popup red line holds: this is the post-run face.&lt;/summary&gt;
void ComposeReportCard()
{
    _reportSpec = ReportCardSpecs.FromStats(NewbieSession.RunCount, _sim.Wave,
        _sim.Kills, _sim.BossCaptures, NewbieSession.TalentState.CodexCollected.Count);
    // T021c-i: ride the build snapshot in the share query (point-card
    // import payload; the receive side parses it back out - T021c-ii).
    // Unmapped picks export NO fragment (honest absence).
    string bd = BuildSnapshotCodec.ExportQuery(_sim.BuildPickIds);
    if (bd.Length &gt; 0) _reportSpec.Query = _reportSpec.Query + "&amp;" + bd;
    _reportCard = new Sy.Social.ShareCardService().Compose(_reportSpec);
    if (_reportCard != null &amp;&amp; _reportCard.Ok) _view.ShowShareOffer();
}
void OnShareRequested()
{
    _view.HideShareOffer();
    // Full three-stage funnel (share_call -&gt; share_ok -&gt; share_return
    // over the local save face); a "" refusal is impossible for a
    // visible button - ComposeReportCard gated it on compose Ok.
    ShareService.CallShare(_reportSpec, _reportCard, _telemetry);
}
// T022-iii: the settle-face friend board entry - ranked rows come
// from the open-data face (mock = save-derived pseudo friends until
// T022-iv); one open = one evt_friend_board funnel beat (K-goal
// instrument: face=settle + the rank/kills the player saw).
```

```
        public SubscribeGrantRow[] subscribeGrants = new SubscribeGrantRow[0];   // T022-ii: one row per type
    }
    /// &lt;summary&gt;
    /// T022-ii: the one-shot subscribe grant ledger row (local save face, red
    /// line 12 - zero server). One row per subscribeId: the day-scoped request
    /// counter drives the 1-type/day anti-nuisance gate (an asked-but-denied
    /// day still burns it); grantedCount accumulates verified grants only.
    /// &lt;/summary&gt;
    [Serializable]
    public class SubscribeGrantRow
    {
        public string subscribeId;
        public int lastRequestDay;   // DayState.DayIndex scope; a new day reopens the gate
        public int requestCount;    // asks fired on lastRequestDay
        public int grantedCount;    // verified grants (mock local face / slice-iv device)
    }
}
// ShareService.cs
using System.Collections.Generic;
namespace Sy.P01.Newbie
{
    /// &lt;summary&gt;
    /// T021b: the three-stage share funnel over the platform bridge. The share
    /// button never dead-ends: the host only raises it after a report card
    /// composed clean, so a visible tap always completes the funnel. The share
    /// return lands on the LOCAL save face (red line 12: zero server - the
    /// shared counter persists in the local slot exactly like every other
    /// out-of-run beat).
    ///   share_call   = the player tapped share (intent, before the bridge).
    ///   share_ok     = the platform share surface accepted the payload.
    ///   share_return = control came back to this process; the local save
    ///                  write completes and the counter persists.
    /// &lt;/summary&gt;
    public static class ShareService
    {
        public const string StageCall = "share_call";
        public const string StageOk = "share_ok";
        public const string StageReturn = "share_return";
        public static bool IsReady =&gt; true;
        /// &lt;summary&gt;One tap = the full funnel. Returns the last fired stage;
        /// "" = honest refusal (null/not-ok inputs, or no platform service) -
        /// the caller hides the button in exactly those states, so a visible
        /// button always completes the funnel (no dead ends).&lt;/summary&gt;
        public static string CallShare(Sy.Social.ShareCard spec,
            Sy.Social.ComposeResult card,
            Sy.Telemetry.TelemetryService telemetry)
        {
            if (spec == null || card == null || !card.Ok) return "";
            if (!Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IPlatformService&gt;(out var platform))
                return "";
```

```
            Track(telemetry, StageCall);
            platform.Share(new Sy.Platform.SharePayload
            {
                Title = spec.Title,
                ImageUrl = card.Path,
                Query = spec.Query,
            });
            Track(telemetry, StageOk);
            // The return beat: the platform sheet closes, control lands back
            // in this process, and the only state change is the local save
            // write (the share return IS a save face - no server round trip).
            NewbieSession.RecordShareReturn();
            Track(telemetry, StageReturn);
            return StageReturn;
        }
        static void Track(Sy.Telemetry.TelemetryService telemetry, string stage)
        {
            if (telemetry == null) return;
            telemetry.Track(Sy.Telemetry.EventIds.Share,
                new Dictionary&lt;string, string&gt; { { "stage", stage } });
        }
    }
}
// SpatialHashGrid.cs
using System;
using System.Collections.Generic;
namespace Sy.P01.Newbie
{
    /// &lt;summary&gt;
    /// Uniform spatial hash grid for same-screen 300-500 enemy collision queries (T008:
    /// spatial hash grid + pure-math circle tests, no Physics2D). Cell size = 2 x contact
    /// range so any circle query spans at most a 3x3 cell block.
    /// Rebuilt from the live ghost list once per tick after movement, before the collision
    /// phases. Pooled per-cell lists keep the steady state allocation-free (T013 GC face).
    /// QueryCircle applies the exact squared-distance test after the cell gather, so its
    /// result set is identical to a linear scan - the tutorial Flow baseline holds.
    /// &lt;/summary&gt;
    public sealed class SpatialHashGrid
    {
        readonly float _minX;
        readonly float _minY;
        readonly float _cellSize;
        readonly int _cols;
        readonly int _rows;
        readonly List&lt;BattleSim.GhostInst&gt;[] _cells;
        public SpatialHashGrid(float minX, float maxX, float minY, float maxY, float cellSize)
        {
            _minX = minX;
            _minY = minY;
            _cellSize = Math.Max(1f, cellSize);
```

```
    _cols = Math.Max(1, (int)Math.Ceiling((maxX - minX) / _cellSize));
    _rows = Math.Max(1, (int)Math.Ceiling((maxY - minY) / _cellSize));
    _cells = new List&lt;BattleSim.GhostInst&gt;[_cols * _rows];
    for (int i = 0; i &lt; _cells.Length; i++) _cells[i] = new List&lt;BattleSim.GhostInst&gt;(16);
}
int ColOf(float x)
{
    int c = (int)((x - _minX) / _cellSize);
    return c &lt; 0 ? 0 : (c &gt;= _cols ? _cols - 1 : c);
}
int RowOf(float y)
{
    int r = (int)((y - _minY) / _cellSize);
    return r &lt; 0 ? 0 : (r &gt;= _rows ? _rows - 1 : r);
}
public void Clear()
{
    for (int i = 0; i &lt; _cells.Length; i++) _cells[i].Clear();
}
public void Insert(BattleSim.GhostInst g)
{
    _cells[RowOf(g.Y) * _cols + ColOf(g.X)].Add(g);
}
/// &lt;summary&gt;O(n) rebuild from the live list; zero allocations after warmup.&lt;/summary&gt;
public void Rebuild(IReadOnlyList&lt;BattleSim.GhostInst&gt; ghosts)
{
    Clear();
    for (int i = 0; i &lt; ghosts.Count; i++) Insert(ghosts[i]);
}
/// &lt;summary&gt;Collects ghosts within radius of (x, y) into the caller-owned buffer.&lt;/summary&gt;
public void QueryCircle(float x, float y, float radius, List&lt;BattleSim.GhostInst&gt; buffer)
{
    buffer.Clear();
    float r2 = radius * radius;
    int c0 = ColOf(x - radius), c1 = ColOf(x + radius);
    int r0 = RowOf(y - radius), r1 = RowOf(y + radius);
    for (int r = r0; r &lt;= r1; r++)
    {
        int rowBase = r * _cols;
        for (int c = c0; c &lt;= c1; c++)
        {
            var cell = _cells[rowBase + c];
            for (int i = 0; i &lt; cell.Count; i++)
            {
                var g = cell[i];
                float dx = g.X - x, dy = g.Y - y;
                if (dx * dx + dy * dy &lt;= r2) buffer.Add(g);
            }
        }
    }
```

```
        }
    }
}
// SubscribeService.cs
using System;
using System.Collections.Generic;
namespace Sy.P01.Newbie
{
    /// &lt;summary&gt;Ask outcome for one subscribe request face call.&lt;/summary&gt;
    public enum SubscribeOutcome
    {
        /// &lt;summary&gt;The player granted the one-shot subscribe request.&lt;/summary&gt;
        Granted,
        /// &lt;summary&gt;The ask fired but was not granted (player denied / unverified
        /// device result) - the ask still counts against the daily cap.&lt;/summary&gt;
        Denied,
        /// &lt;summary&gt;No ask ever reached the player (daily gate block, or the
        /// platform face refused pre-dialog) - zero events, zero ledger burn.&lt;/summary&gt;
        NotAsked,
        /// &lt;summary&gt;The subscribeId is not in cfg_subscribe (fail loud, no
        /// fabricated asks - the data layer is the only type authority).&lt;/summary&gt;
        UnknownType
    }
    /// &lt;summary&gt;
    /// T022-ii: the subscribe request face swap point (T001c paradigm). The
    /// local mock face is authoritative until the real account-side WX template
    /// ids arrive (U014); the armed-build WX face swaps in without touching
    /// callers. Zero shared-package changes by design (P02/P06 pending-review
    /// byte stability, X236 law).
    /// &lt;/summary&gt;
    public interface ISubscribeFace
    {
        /// &lt;summary&gt;Fire the ask for one cfg_subscribe row. NotAsked = the face
        /// never asked (placeholder template ids refuse with zero device calls).
        /// onDeviceResult = the T022-iv-B result channel: an armed async face
        /// may deliver the verified device grant later (true) - the grant only
        /// ever lands through that callback, never the sync return. Faces
        /// without a device result (mock faces grant synchronously) never
        /// touch it.&lt;/summary&gt;
        SubscribeOutcome Request(P01Configs.SubscribeRow row, Action&lt;bool&gt; onDeviceResult);
    }
    /// &lt;summary&gt;
    /// T022-ii: the one-shot subscribe request service, four cfg-driven faces
    /// (codex_new_ghost / endless_record / share_return / night_ready per D008).
    /// Red line 12: zero server - the grant ledger lives in the LOCAL save slot
    /// exactly like the share-return beat (the ask IS a save face). The
    /// anti-nuisance gate = 1 request per type per day (cfg requestCapPerDay,
    /// R38a-pinned); an asked-but-denied day still burns the gate.
    /// Funnel: evt_subscribe stage=request (the ask fired) then stage=granted
    /// (verified grant only). Gate-blocked and pre-dialog refusals fire zero
```

```
/// events (the player was never asked).
/// &lt;/summary&gt;
public static class SubscribeService
{
    public const string EventId = "evt_subscribe";
    public const string StageRequest = "request";
    public const string StageGranted = "granted";
    /// &lt;summary&gt;Placeholder template-id prefix (cfg swap face): ids with
    /// this prefix are not real console ids and never reach the device.&lt;/summary&gt;
    public const string PlaceholderPrefix = "placeholder_tmpl_";
    // The four live cfg_subscribe ids (D008; stamina_full stays a reserved
    // object - no trigger source may be fabricated for it).
    public const string CodexNewGhost = "codex_new_ghost";
    public const string EndlessRecord = "endless_record";
    public const string ShareReturn = "share_return";
    public const string NightReady = "night_ready";
    static ISubscribeFace _faceOverride;
    /// &lt;summary&gt;Editor/gate injection point; also the channel-suite seam
    /// for future platform wiring (MockLaunchInfo paradigm).&lt;/summary&gt;
    public static void SetFace(ISubscribeFace face) { _faceOverride = face; }
    /// &lt;summary&gt;The active ask face: gate override first, then the channel
    /// default (armed WX builds swap the real face in, X129 law).&lt;/summary&gt;
    public static ISubscribeFace Face =&gt; _faceOverride ?? DefaultFaceFactory.Create();
    /// &lt;summary&gt;One ask for one cfg_subscribe type. The full chain: type
    /// lookup -&gt; daily gate -&gt; face ask -&gt; ledger write (local save) -&gt;
    /// funnel events. Never dead-ends: every call returns a definite
    /// outcome and the caller can always surface it.&lt;/summary&gt;
    public static SubscribeOutcome Subscribe(string subscribeId,
        Sy.Telemetry.TelemetryService telemetry)
    {
        var row = FindRow(subscribeId);
        if (row == null) return SubscribeOutcome.UnknownType;
        int day = NewbieSession.DayState.DayIndex;
        int cap = row.requestCapPerDay &gt; 0 ? row.requestCapPerDay : 1;
        SubscribeGrantRow grant = null;
        for (int i = 0; i &lt; NewbieSession.SubscribeGrants.Count; i++)
        {
            var g = NewbieSession.SubscribeGrants[i];
            if (g != null &amp;&amp; g.subscribeId == subscribeId) { grant = g; break; }
        }
        if (grant != null &amp;&amp; grant.lastRequestDay == day &amp;&amp; grant.requestCount &gt;= cap)
            return SubscribeOutcome.NotAsked;   // daily gate: silent block
        var outcome = Face.Request(row, granted =&gt; RecordDeviceGrant(row, granted, telemetry));
        if (outcome == SubscribeOutcome.NotAsked)
            return SubscribeOutcome.NotAsked;   // pre-dialog refusal: no ledger burn
        Track(telemetry, StageRequest, row);
        if (grant == null)
        {
            grant = new SubscribeGrantRow { subscribeId = subscribeId };
            NewbieSession.SubscribeGrants.Add(grant);
```

```
    }
    if (grant.lastRequestDay != day)
    {
        grant.lastRequestDay = day;   // new day: the counter reopens
        grant.requestCount = 0;
    }
    grant.requestCount++;
    if (outcome == SubscribeOutcome.Granted) grant.grantedCount++;
    NewbieSession.WriteToSave();   // the ask beat is a save face (share-return law)
    if (outcome == SubscribeOutcome.Granted)
        Track(telemetry, StageGranted, row);
    return outcome;
}
// ── The four request faces (cfg_subscribe trigger.source documents the
//    may-ask moments: RegisterCodexCapture / BeginEndlessRun /
//    RecordShareReturn / ChainRewardClaimed - the hosts call the face
//    matching their beat when the ask-moment UI lands). ──
public static SubscribeOutcome RequestCodexNewGhost(Sy.Telemetry.TelemetryService telemetry)
    =&gt; Subscribe(CodexNewGhost, telemetry);
public static SubscribeOutcome RequestEndlessRecord(Sy.Telemetry.TelemetryService telemetry)
    =&gt; Subscribe(EndlessRecord, telemetry);
public static SubscribeOutcome RequestShareReturn(Sy.Telemetry.TelemetryService telemetry)
    =&gt; Subscribe(ShareReturn, telemetry);
public static SubscribeOutcome RequestNightReady(Sy.Telemetry.TelemetryService telemetry)
    =&gt; Subscribe(NightReady, telemetry);
/// &lt;summary&gt;cfg row lookup or null (the data layer is the only type
/// authority - unknown ids fail loud, nothing is fabricated).&lt;/summary&gt;
public static P01Configs.SubscribeRow FindRow(string subscribeId)
{
    var table = P01Configs.Subscribe;
    if (table == null || table.types == null) return null;
    foreach (var t in table.types)
        if (t != null &amp;&amp; t.subscribeId == subscribeId) return t;
    return null;
}
/// &lt;summary&gt;T022-iv-B: the device result channel's grant landing (the
/// armed WX real-id era). The sync face returns Denied as the honest
/// pending outcome; this lands later, only on a verified device grant:
/// grantedCount bumps, the granted funnel event fires and the ledger
/// re-saves through the same local slot. A denied device result lands
/// nothing (the ask already burned as Denied on the sync path).&lt;/summary&gt;
static void RecordDeviceGrant(P01Configs.SubscribeRow row, bool granted,
    Sy.Telemetry.TelemetryService telemetry)
{
    if (!granted) return;
    var grant = FindGrantRow(row.subscribeId);
    if (grant == null) return;   // nothing was ever asked - never fabricate
    grant.grantedCount++;
    NewbieSession.WriteToSave();
    Track(telemetry, StageGranted, row);
```

```
        }
        /// &lt;summary&gt;The ledger row for one subscribe id, or null (device-result
        /// grants only ever land on a row the ask beat already wrote).&lt;/summary&gt;
        public static SubscribeGrantRow FindGrantRow(string subscribeId)
        {
            for (int i = 0; i &lt; NewbieSession.SubscribeGrants.Count; i++)
            {
                var g = NewbieSession.SubscribeGrants[i];
                if (g != null &amp;&amp; g.subscribeId == subscribeId) return g;
            }
            return null;
        }
        static void Track(Sy.Telemetry.TelemetryService telemetry, string stage,
            P01Configs.SubscribeRow row)
        {
            if (telemetry == null) return;
            telemetry.Track(EventId, new Dictionary&lt;string, string&gt;
            {
                { "stage", stage },
                { "subscribeId", row.subscribeId },
                { "category", row.@class },
            });
        }
    }
    /// &lt;summary&gt;
    /// The local authoritative ask face (Mock law, T001c): simulates the player
    /// granting the request. Editor, batchmode gates and SY_MOCK builds run on
    /// this face until the real account-side template ids arrive (U014).
    /// &lt;/summary&gt;
    public sealed class MockSubscribeFace : ISubscribeFace
    {
        public SubscribeOutcome Request(P01Configs.SubscribeRow row, Action&lt;bool&gt; onDeviceResult)
            =&gt; SubscribeOutcome.Granted;   // sync local grant - no device result to deliver
    }
    /// &lt;summary&gt;Channel default resolution (X129 law: the WX DllImport surface
    /// only exists on armed weixin builds; everything else = local face).&lt;/summary&gt;
    static class DefaultFaceFactory
    {
#if SY_WECHAT
        public static ISubscribeFace Create() =&gt; new WxSubscribeFace();
#else
        public static ISubscribeFace Create() =&gt; new MockSubscribeFace();
#endif
    }
}
// TalentLauncher.cs
using System.Collections.Generic;
using UnityEngine;
namespace Sy.P01.Newbie
{
```

```
/// &lt;summary&gt;
/// T018 slice 3: S01 out-of-run talent/codex host (pitch s3). Hosted by MainUI
/// at Awake AFTER the DayLauncher (AddComponent pattern - no scene YAML
/// churn; later sibling = the entry button renders above the day panel).
/// Owns the hub face of the talent tree: the pinned TalentSim session buys
/// points (wallet mirror deducts; the host reads SpendCoins back so the
/// session books stay authoritative - DaySim precedent), the branch/codex
/// panel displays the collection. The chain claim verb is NOT here: the
/// won-night settle auto-claims (NewbieSession.DepositNight), this panel
/// makes the grant visible. Funnel telemetry rides evt_talent_layer
/// (open/buy/deny/close + day).
/// T035 slice 6 (X2099): every visible label resolves through UiText -&gt;
/// cfg_i18n (the shipped table copy is the only place CJK lives).
/// &lt;/summary&gt;
public sealed class TalentLauncher : MonoBehaviour
{
    TalentSim _sim;
    TalentView _view;
    Sy.Telemetry.TelemetryService _telemetry;
    void Awake()
    {
        var canvas = FindObjectOfType&lt;Canvas&gt;();
        _sim = NewbieSession.TalentSimToday;
        _view = gameObject.AddComponent&lt;TalentView&gt;();
        _view.Initialize(canvas);
        _telemetry = new Sy.Telemetry.TelemetryService();
        _telemetry.AddSink(new TelemetryDebugSink());
        _view.OpenRequested += OnOpenRequested;
        _view.BuyRequested += OnBuyRequested;
        _view.CloseRequested += OnCloseRequested;
        _view.CodexPageRequested += OnCodexPageRequested;
        _view.TalentPageRequested += OnTalentPageRequested;
        _sim.PointBought += OnPointBought;
        _sim.BuyDenied += OnBuyDenied;
    }
    void OnOpenRequested()
    {
        RefreshPanel();
        Track("open");
    }
    void OnTalentPageRequested()
    {
        RefreshPanel();
        Track("page", "talent");
    }
    void OnCodexPageRequested()
    {
        RefreshCodexPanel();
        Track("page", "codex");
    }
```

```
void RefreshPanel()
{
    var branches = BuildBranchFaces();
    _view.ShowTalent(NewbieSession.TalentState.DayIndex, NewbieSession.WalletCoins,
        branches, _sim.NextPointCost(), ChainStatus(), NewbieSession.TalentState.WeekClaimed);
}
/// &lt;summary&gt;T019 slice 3: the codex page refresh - the roster from cfg_enemy
/// (row order = bookSlot order), lit per the session collection, the live
/// global bonus from CollectionBook (the panel never re-derives it).&lt;/summary&gt;
void RefreshCodexPanel()
{
    var faces = BuildCodexFaces();
    int count = NewbieSession.TalentState.CodexCollected.Count;
    var coll = P01Configs.Collection;
    int total = coll != null &amp;&amp; coll.collection != null ? coll.collection.firstVersionCount : 10;
    _view.ShowCodex(NewbieSession.TalentState.DayIndex, NewbieSession.WalletCoins, faces,
        count, total, Newbie.CollectionBook.GlobalPctFor(count), ChainStatus());
}
TalentView.CodexSlotFace[] BuildCodexFaces()
{
    var enemy = P01Configs.Enemy;
    var rows = enemy != null &amp;&amp; enemy.templates != null ? enemy.templates : null;
    var faces = new TalentView.CodexSlotFace[rows != null ? rows.Length : 0];
    for (int i = 0; i &lt; faces.Length; i++)
    {
        var t = rows[i];
        faces[i] = new TalentView.CodexSlotFace
        {
            CodeName = t.codeName,
            NameCn = SpeciesNameCn(t.enemyId),
            Collected = NewbieSession.TalentState.CodexCollected.Contains(t.enemyId),
        };
    }
    return faces;
}
// T035 slice 6: the first-version CN display names moved to the loc table
// keyed by enemy id (talent_codex_&lt;id&gt;); cfg_enemy carries nameKey loc ids
// and the nameKey face lands with the codex art batch (T026). Unknown ids
// keep the raw id (the TryGetValue-miss kin; a table miss returns the key).
static string SpeciesNameCn(string enemyId)
{
    string key = "talent_codex_" + enemyId;
    string name = UiText.Get(key);
    return name != key ? name : enemyId;
}
void OnBuyRequested(string branchId)
{
    var reason = _sim.BuyPoint(branchId);
    if (reason != TalentSim.BuyDenyReason.None) Track("deny", reason.ToString());
}
```

```
        void OnPointBought(string branchId, int newPoints, int cost)
        {
            // Host books stay authoritative: deduct the talent session's spend counter.
            NewbieSession.SpendFromTalent(_sim.SpendCoins);
            NewbieSession.WriteToSave();   // T019 slice 2: the buy persists immediately
            RefreshPanel();
            _view.PlayBuyFlash(BranchName(branchId));
            Track("buy", branchId);
        }
        void OnBuyDenied(TalentSim.BuyDenyReason reason)
        {
            switch (reason)
            {
                case TalentSim.BuyDenyReason.BranchNotFound:
                    _view.SetHint(UiText.Get("talent_deny_branch"));
                    break;
                case TalentSim.BuyDenyReason.BranchFull:
                    _view.SetHint(UiText.Get("talent_deny_full"));
                    break;
                case TalentSim.BuyDenyReason.DailyCapReached:
                    _view.SetHint(UiText.Get("talent_deny_cap"));
                    break;
                case TalentSim.BuyDenyReason.NotEnoughCoins:
                    _view.SetHint(UiText.Get("talent_deny_coins_fmt", _sim.NextPointCost().ToString()));
                    break;
                default:
                    _view.SetHint(UiText.Get("talent_deny_closed"));
                    break;
            }
        }
        void OnCloseRequested()
        {
            _view.Hide();
            Track("close");
        }
        TalentView.BranchFace[] BuildBranchFaces()
        {
            var cfg = P01Configs.Talent;
            var rows = cfg != null &amp;&amp; cfg.tree != null &amp;&amp; cfg.tree.branches != null ? cfg.tree.branches
            var faces = new TalentView.BranchFace[rows != null ? rows.Length : 0];
            for (int i = 0; i &lt; faces.Length; i++)
            {
                var b = rows[i];
                faces[i] = new TalentView.BranchFace
                {
                    Id = b.branchId,
                    Name = b.name,
                    Points = NewbieSession.TalentState.PointsIn(b.branchId),
                    Max = b.points,
                    NextMilestoneDesc = NextMilestoneDesc(b),
```

```
        };
    }
    return faces;
}
static string NextMilestoneDesc(P01Configs.TalentBranchRow branch)
{
    if (branch.milestones == null || branch.milestones.Length == 0) return "";
    int pts = NewbieSession.TalentState.PointsIn(branch.branchId);
    for (int i = 0; i &lt; branch.milestones.Length; i++)
        if (branch.milestones[i].atPoints &gt; pts)
            return UiText.Get("talent_next_node_fmt", branch.milestones[i].atPoints.ToString(),
                branch.milestones[i].desc);
    return UiText.Get("talent_branch_full");
}
string ChainStatus()
{
    var row = _sim.ChainDayToday();
    if (row == null || row.rewards == null)
    {
        int collected = NewbieSession.TalentState.CodexCollected.Count;
        return UiText.Get("talent_chain_done_fmt", collected.ToString());
    }
    return UiText.Get("talent_chain_active_fmt", row.dayIndex.ToString(), row.rewards.enemyId,
        row.rewards.coins.ToString());
}
static string BranchName(string branchId)
{
    var cfg = P01Configs.Talent;
    var rows = cfg != null &amp;&amp; cfg.tree != null &amp;&amp; cfg.tree.branches != null ? cfg.tree.branches
    if (rows == null) return branchId;
    for (int i = 0; i &lt; rows.Length; i++)
        if (rows[i].branchId == branchId) return rows[i].name;
    return branchId;
}
void Track(string state, string detail = null)
{
    // T024-ii: delegated to the shared seam (LauncherTelemetry) so the
    // marathon gate rides the exact same field construction; zero
    // behavior change here - same fields, same id, same order.
    LauncherTelemetry.TrackTalent(_telemetry, state, detail, NewbieSession.TalentState.DayIndex);
}
sealed class TelemetryDebugSink : Sy.Telemetry.TelemetryService.ISink
{
    public void Write(string evtId, Dictionary&lt;string, string&gt; fields)
    {
        var sb = new System.Text.StringBuilder(evtId);
        foreach (var kv in fields) sb.Append(' ').Append(kv.Key).Append('=').Append(kv.Value);
        Debug.Log("[TELEMETRY] " + sb);
    }
}
```

```
    }
}
// TalentSim.cs
using System;
using System.Collections.Generic;
namespace Sy.P01.Newbie
{
    /// &lt;summary&gt;
    /// T018 slice 2b: the out-of-run talent session state machine (pure C#, zero
    /// rendering, zero rng, zero server - deterministic by construction, red line
    /// 12 safe). Two faces over cfg_talent:
    /// (a) the 30-point tree - buy ONE point per day (dailyBuyCap=1, the pace gate
    ///     independent of coins) at the GLOBAL curve cost 55n+45 where n is the
    ///     global point index (P1=100 -&gt; P30=1695, total 26,925 &lt;= the 30,000 no-ads
    ///     D30 disposable = the 30-day closure); progression is linear inside a
    ///     branch, no cross-branch gates (free orthogonal builds);
    /// (b) the seven-day codex chain - each session day D1-D7 the host reports won
    ///     nights; one claim per day grants that day's codex ghost (slot = dayIndex,
    ///     roster order) + chain coins 20..80 (total 350 &lt;= the 1,075 economy buffer
    ///     - a retention hook, never an income pillar).
    /// The wallet mirrors the host-injected coin snapshot; the host reads the spend
    /// and gain counters back and owns the authoritative books (DaySim precedent).
    /// BattleSim is untouched by construction, so the Flow baseline has zero drift
    /// surface. The talent/codex UI + session wiring land in slice 3.
    /// Authoring authority = Design/configs/P01/cfg_talent.json (05 s3 + s8).
    /// &lt;/summary&gt;
    public sealed class TalentSim
    {
        public enum BuyDenyReason
        {
            None = 0,
            BranchNotFound,
            BranchFull,
            DailyCapReached,
            NotEnoughCoins
        }
        public enum ChainDenyReason
        {
            None = 0,
            NotInChainWindow,
            TaskIncomplete,
            AlreadyClaimed
        }
        readonly TalentState _state;
        readonly P01Configs.TalentTableFile _cfg;
        public TalentSim(TalentState state, P01Configs.TalentTableFile cfg, int coins)
        {
            _state = state ?? new TalentState();
            _cfg = cfg;
            WalletCoins = coins;
```

```
}
// Wallet mirror + host readback counters.
public int WalletCoins { get; private set; }
public int SpendCoins { get; private set; }
public int ChainCoinsGained { get; private set; }
P01Configs.TalentCostCurveBlock Curve
{
    get { return _cfg != null &amp;&amp; _cfg.tree != null ? _cfg.tree.costCurve : null; }
}
public event Action&lt;string, int, int&gt; PointBought;              // (branchId, newPointsInBranch, cost)
public event Action&lt;BuyDenyReason&gt; BuyDenied;
public event Action&lt;int&gt; NightWinReported;                       // (winsToday)
public event Action&lt;int, string, int, int&gt; ChainRewardClaimed;   // (dayIndex, enemyId, coins, codexSlot)
public event Action&lt;ChainDenyReason&gt; ChainClaimDenied;
/// &lt;summary&gt;Global curve cost of the next point: 55n+45 with n = global point
/// index 1..30 (P1=100, P30=1695; R27 pins the recomputed sum 26,925).&lt;/summary&gt;
public int NextPointCost()
{
    var curve = Curve;
    if (curve == null) return int.MaxValue;
    return curve.slopePerPoint * (_state.TotalPoints + 1) + curve.baseOffset;
}
/// &lt;summary&gt;Buy one point in a branch (linear in-branch progression: the next
/// point is always current points + 1). Deny precedence: branch not found -&gt;
/// branch full -&gt; daily cap -&gt; not enough coins (checked in this order).&lt;/summary&gt;
public BuyDenyReason BuyPoint(string branchId)
{
    _state.EnsureDay();
    var branch = FindBranch(branchId);
    if (branch == null) return DenyBuy(BuyDenyReason.BranchNotFound);
    int inBranch = _state.PointsIn(branchId);
    if (inBranch &gt;= branch.points) return DenyBuy(BuyDenyReason.BranchFull);
    var curve = Curve;
    if (curve != null &amp;&amp; _state.BuysToday &gt;= curve.dailyBuyCap)
        return DenyBuy(BuyDenyReason.DailyCapReached);
    int cost = NextPointCost();
    if (WalletCoins &lt; cost) return DenyBuy(BuyDenyReason.NotEnoughCoins);
    WalletCoins -= cost;
    SpendCoins += cost;
    _state.BuysToday++;
    _state.AddPoint(branchId);
    PointBought?.Invoke(branchId, inBranch + 1, cost);
    return BuyDenyReason.None;
}
/// &lt;summary&gt;Host reports a won night (the chain verb: win 1 night/day).&lt;/summary&gt;
public void ReportNightWin()
{
    _state.EnsureDay();
    _state.NightWinsToday++;
    NightWinReported?.Invoke(_state.NightWinsToday);
```

```
}
/// &lt;summary&gt;Today's chain row (null outside the D1-D7 window or table missing).&lt;/summary&gt;
public P01Configs.WeekChainDayRow ChainDayToday()
{
    if (_cfg == null || _cfg.weekChain == null || _cfg.weekChain.days == null) return null;
    var days = _cfg.weekChain.days;
    int d = _state.DayIndex;
    for (int i = 0; i &lt; days.Length; i++)
        if (days[i].dayIndex == d) return days[i];
    return null;
}
/// &lt;summary&gt;Claim today's chain reward once the day's win requirement holds:
/// grants the codex ghost (slot = dayIndex, roster order) + chain coins into
/// the wallet. Deny precedence: outside window -&gt; already claimed -&gt; task not
/// complete yet.&lt;/summary&gt;
public ChainDenyReason ClaimChainDay()
{
    _state.EnsureDay();
    var row = ChainDayToday();
    if (row == null) return DenyChain(ChainDenyReason.NotInChainWindow);
    int slot = _state.DayIndex - 1;
    if (slot &gt;= 0 &amp;&amp; slot &lt; _state.WeekClaimed.Length &amp;&amp; _state.WeekClaimed[slot])
        return DenyChain(ChainDenyReason.AlreadyClaimed);
    int need = row.require != null ? row.require.winNights : 1;
    if (_state.NightWinsToday &lt; need) return DenyChain(ChainDenyReason.TaskIncomplete);
    if (slot &gt;= 0 &amp;&amp; slot &lt; _state.WeekClaimed.Length) _state.WeekClaimed[slot] = true;
    int coins = row.rewards != null ? row.rewards.coins : 0;
    string enemyId = row.rewards != null ? row.rewards.enemyId : null;
    int codexSlot = row.rewards != null ? row.rewards.codexSlot : 0;
    WalletCoins += coins;
    ChainCoinsGained += coins;
    // T019 slice 3: dedupe against the capture suspender - a species the
    // player already capture-registered never double-counts when its chain
    // day arrives (belt + suspenders, one entry per species).
    if (!string.IsNullOrEmpty(enemyId) &amp;&amp; !_state.CodexCollected.Contains(enemyId))
        _state.CodexCollected.Add(enemyId);
    ChainRewardClaimed?.Invoke(_state.DayIndex, enemyId, coins, codexSlot);
    return ChainDenyReason.None;
}
// ── derived stat face: the host reads these when wiring the in-run effects
// (BattleSim consumption lands with a later slice; nothing there changes now).
P01Configs.TalentBranchRow FindBranch(string branchId)
{
    if (_cfg == null || _cfg.tree == null || _cfg.tree.branches == null) return null;
    var branches = _cfg.tree.branches;
    for (int i = 0; i &lt; branches.Length; i++)
        if (branches[i].branchId == branchId) return branches[i];
    return null;
}
P01Configs.TalentPerPointBlock PerPointOf(string branchId)
```

```
{
    var b = FindBranch(branchId);
    return b != null ? b.perPoint : null;
}
/// &lt;summary&gt;Aggregated per-point attack bonus (soul: +2%/pt, full +20%).&lt;/summary&gt;
public float AtkPct
{
    get
    {
        var pp = PerPointOf("soul");
        return pp != null ? pp.atkPct * _state.SoulPoints : 0f;
    }
}
/// &lt;summary&gt;Multiplicative skill-CD multiplier (rune: 0.97^pts, ~26% full).&lt;/summary&gt;
public double SkillCdMul
{
    get
    {
        var pp = PerPointOf("rune");
        return pp != null ? Math.Pow(1.0 + pp.cdPct, _state.RunePoints) : 1.0;
    }
}
/// &lt;summary&gt;Aggregated per-point HP bonus (body: +3%/pt, full +30%).&lt;/summary&gt;
public float HpPct
{
    get
    {
        var pp = PerPointOf("body");
        return pp != null ? pp.hpPct * _state.BodyPoints : 0f;
    }
}
/// &lt;summary&gt;Milestone lookup: the branch milestone's effect-field value once
/// branch points reach atPoints (each field is unique per branch, so the
/// highest unlocked milestone carrying a non-zero value wins; 0 when none).&lt;/summary&gt;
float Milestone(string branchId, Func&lt;P01Configs.TalentMilestoneEffectBlock, float&gt; field)
{
    var b = FindBranch(branchId);
    if (b == null || b.milestones == null) return 0f;
    int pts = _state.PointsIn(branchId);
    float v = 0f;
    for (int i = 0; i &lt; b.milestones.Length; i++)
    {
        var m = b.milestones[i];
        if (m == null || m.effect == null || m.atPoints &gt; pts) continue;
        float fv = field(m.effect);
        if (fv != 0f) v = fv;
    }
    return v;
}
public float KillSoulRegenPerSec { get { return Milestone("soul", e =&gt; e.killSoulRegenPerSec); } }
```

```
        public float SoulFireFamilyDmgPct { get { return Milestone("soul", e =&gt; e.soulFireFamilyDmgPct); } }
        public int InRunRerollCharges { get { return (int)Milestone("rune", e =&gt; e.inRunRerollCharges); } }
        public float EvolveThresholdPct { get { return Milestone("rune", e =&gt; e.evolveThresholdPct); } }
        public float MoveSpeedPct { get { return Milestone("body", e =&gt; e.moveSpeedPct); } }
        public int ReviveInvulnSec { get { return (int)Milestone("body", e =&gt; e.reviveInvulnSec); } }
        public int DeathShieldCharges { get { return (int)Milestone("body", e =&gt; e.deathShieldCharges); } }
        BuyDenyReason DenyBuy(BuyDenyReason reason)
        {
            BuyDenied?.Invoke(reason);
            return reason;
        }
        ChainDenyReason DenyChain(ChainDenyReason reason)
        {
            ChainClaimDenied?.Invoke(reason);
            return reason;
        }
    }
}
// TalentState.cs
using System.Collections.Generic;
namespace Sy.P01.Newbie
{
    /// &lt;summary&gt;
    /// T018 slice 2b: cross-day persistent state for the out-of-run talent tree and
    /// the seven-day codex chain (pitch s3, 05 s3 + s8). Plain C# POCO, zero
    /// UnityEngine dependency, zero server (red line 12) - the host owns the instance
    /// and the local save face lands with T019, so nothing here serializes yet by
    /// design (DayState precedent).
    /// DayIndex is 1-based and shares the DayState domain (the player's session day).
    /// &lt;/summary&gt;
    public sealed class TalentState
    {
        public int DayIndex = 1;
        // Branch points bought (soul/rune/body, 0..10 each per cfg branches[].points).
        public int SoulPoints;
        public int RunePoints;
        public int BodyPoints;
        // Daily buy cap latch (cfg costCurve.dailyBuyCap = 1; resets on day change).
        public int BuysToday;
        public int BuyDayIndex = 1;
        // Night-win counter for the week-chain requirement (win 1 night/day).
        public int NightWinsToday;
        public int WinsDayIndex = 1;
        // Week chain D1-D7 claim latch + the codex collection the chain grants.
        public bool[] WeekClaimed = new bool[7];
        public readonly List&lt;string&gt; CodexCollected = new List&lt;string&gt;();
        public int TotalPoints { get { return SoulPoints + RunePoints + BodyPoints; } }
        public int PointsIn(string branchId)
        {
            switch (branchId)
```

```
            {
                case "soul": return SoulPoints;
                case "rune": return RunePoints;
                case "body": return BodyPoints;
                default: return -1;
            }
        }
        public void AddPoint(string branchId)
        {
            switch (branchId)
            {
                case "soul": SoulPoints++; break;
                case "rune": RunePoints++; break;
                case "body": BodyPoints++; break;
            }
        }
        /// &lt;summary&gt;Day rollover: the host advances DayIndex directly; this syncs the
        /// per-day counters to the current day so caps reset on a new session day.&lt;/summary&gt;
        public void EnsureDay()
        {
            if (BuyDayIndex != DayIndex) { BuysToday = 0; BuyDayIndex = DayIndex; }
            if (WinsDayIndex != DayIndex) { NightWinsToday = 0; WinsDayIndex = DayIndex; }
        }
    }
}
// TalentView.cs
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.P01.Newbie
{
    /// &lt;summary&gt;
    /// T018 slice 3: the S01 out-of-run talent/codex panel (pitch s3). Pure
    /// programmatic UGUI mirroring the DayView factory style (no scene YAML
    /// churn, no art dependency - headless safe). Violet meta palette to read
    /// as "soul ledger" against the day panel's warm gold. Faces:
    /// - a corner entry button ("talent/codex") always visible in the hub,
    ///   rendered above the day panel (created later in MainUI.Awake order);
    /// - the panel: three branch rows (name/points/next cost + buy button,
    ///   next milestone desc from cfg), the seven codex slots D1-D7, today's
    ///   chain status line, wallet label and a close button;
    /// - T019 slice 3: a codex PAGE inside the same panel (corner switch) - the
    ///   10-species roster grid lighting per the collection + the live global
    ///   bonus progress line (the capture-&gt;codex-&gt;craft loop's visible face).
    /// The claim verb itself is NOT here: the deposit auto-claims at the won
    /// night's settle (zero-friction retention hook), this panel makes the
    /// collection and the next milestone visible (shareable collection face).
    /// Copy source: task book T018 (talent tree 30 pts / codex slots D1-D7 /
```

```
/// chain win-claim UI) + T019 (CollectionBook codex page) + cfg_talent
/// branches + weekChain days.
/// T035 slice 5 (X2095): every visible label resolves through UiText -&gt;
/// cfg_i18n (talent_* 16 keys; branch row {0}-{3}, codex progress {0}/{1}/{2},
/// day/wallet fmts), rendered byte-identical to the pre-migration literals,
/// so the source file carries zero CJK string literals (I18nLintCheck L4).
/// &lt;/summary&gt;
public sealed class TalentView : MonoBehaviour
{
    /// &lt;summary&gt;One branch row's display face (built by the launcher from cfg + state).&lt;/summary&gt;
    public struct BranchFace
    {
        public string Id;
        public string Name;
        public string NextMilestoneDesc;
        public int Points;
        public int Max;
    }
    Canvas _canvas;
    Sprite _square;
    RectTransform _panel;
    Text _dayLabel, _walletText, _hintText, _chainText, _titleText, _codexProgressText, _codexBeltText;
    AudioSource _toneSource;
    readonly List&lt;string&gt; _branchIds = new();
    readonly List&lt;Text&gt; _slotTexts = new();
    readonly List&lt;Text&gt; _codexSlotTexts = new();
    // T019 slice 3: page sets inside the one talent/codex panel - the talent page
    // (branch rows + week-chain slots) and the codex page (10-species roster).
    readonly List&lt;GameObject&gt; _talentPageObjects = new();
    readonly List&lt;GameObject&gt; _codexPageObjects = new();
    public event Action OpenRequested;
    public event Action&lt;string&gt; BuyRequested;
    public event Action CloseRequested;
    public event Action CodexPageRequested;
    public event Action TalentPageRequested;
    // Meta palette (violet soul-ledger vs day gold vs battle neon dark).
    static readonly Color Violet = new Color(0.62f, 0.44f, 1f);
    static readonly Color MetaBg = new Color(0.07f, 0.05f, 0.12f, 0.97f);
    static readonly Color CardDim = new Color(0.16f, 0.12f, 0.24f);
    static readonly Color BoneWhite = new Color(0.941f, 0.929f, 0.902f);
    static readonly Color NeonCyan = new Color(0f, 1f, 0.82f);
    public void Initialize(Canvas canvas)
    {
        _canvas = canvas;
        _square = MakeSquareSprite();
        BuildEntryButton();
        BuildTalentPanel();
    }
    void BuildEntryButton()
    {
```

```
        var go = new GameObject("TalentEntry", typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(_canvas.transform, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = new Vector2(390, 850);
        rt.sizeDelta = new Vector2(260, 96);
        var img = go.GetComponent&lt;Image&gt;();
        img.sprite = _square;
        img.color = new Color(0.30f, 0.20f, 0.48f, 0.92f);
        MakeText(go.transform, UiText.Get("talent_entry"), 40, Vector2.zero, TextAnchor.MiddleCenter, 260, 96).color
        go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; OpenRequested?.Invoke());
    }
    void BuildTalentPanel()
    {
        _panel = MakeOverlayPanel("TalentPanel", MetaBg);
        _titleText = MakeText(_panel, UiText.Get("talent_title"), 84, new Vector2(0, 620), TextAnchor.MiddleCenter,
        _titleText.color = Violet;
        _dayLabel = MakeText(_panel, UiText.Get("talent_day_fmt", "1"), 44, new Vector2(0, 520), TextAnchor.MiddleCe
        _dayLabel.color = BoneWhite;
        _walletText = MakeText(_panel, "", 40, new Vector2(0, 445), TextAnchor.MiddleCenter, 1040, 70);
        _walletText.color = NeonCyan;
        // ── talent page (T018): branch rows + week-chain slots ──
        for (int i = 0; i &lt; 3; i++)
        {
            int row = i;
            var card = new GameObject("BranchRow" + row, typeof(RectTransform), typeof(Image));
            card.transform.SetParent(_panel, false);
            var crt = (RectTransform)card.transform;
            crt.anchorMin = crt.anchorMax = new Vector2(0.5f, 0.5f);
            crt.anchoredPosition = new Vector2(0, 290 - row * 150);
            crt.sizeDelta = new Vector2(1040, 130);
            var cimg = card.GetComponent&lt;Image&gt;();
            cimg.sprite = _square;
            cimg.color = CardDim;
            // texts[0]=main, texts[1]=milestone desc (filled per ShowTalent).
            MakeText(card.transform, "", 42, new Vector2(-110, 22), TextAnchor.MiddleLeft, 620, 60).color = BoneWhit
            MakeText(card.transform, "", 32, new Vector2(-110, -36), TextAnchor.MiddleLeft, 620, 56).color = new Col
            MakeButtonOn(card.transform, UiText.Get("talent_buy"), new Vector2(400, 0), new Vector2(200, 100),
                new Color(0.45f, 0.30f, 0.72f), () =&gt; BuyRequested?.Invoke(_branchIds[row]));
            _talentPageObjects.Add(card);
        }
        _talentPageObjects.Add(MakeText(_panel, UiText.Get("talent_chain_foot"), 38, new Vector2(0, -195),
            TextAnchor.MiddleCenter, 1040, 70).gameObject);
        for (int i = 0; i &lt; 7; i++)
        {
            var slot = MakeText(_panel, "D" + (i + 1), 40, new Vector2(-390 + i * 130, -300),
                TextAnchor.MiddleCenter, 120, 100);
            slot.color = new Color(0.35f, 0.30f, 0.45f);
            _slotTexts.Add(slot);
            _talentPageObjects.Add(slot.gameObject);
```

```
        }
        _chainText = MakeText(_panel, "", 36, new Vector2(0, -420), TextAnchor.MiddleCenter, 1040, 80);
        _chainText.color = new Color(0.85f, 0.80f, 0.95f);
        _talentPageObjects.Add(_chainText.gameObject);
        // Corner page switch (talent page -&gt; codex page): the collection is one
        // tap away, no second hub button (one panel owns both codex faces).
        _talentPageObjects.Add(MakeButtonOn(_panel, UiText.Get("talent_to_codex"), new Vector2(430, 620), new Vector
            new Color(0.30f, 0.42f, 0.55f, 0.95f), () =&gt; CodexPageRequested?.Invoke()).gameObject);
        // ── codex page (T019 slice 3): the 10-species roster + bonus progress ──
        _codexProgressText = MakeText(_panel, "", 48, new Vector2(0, 290),
            TextAnchor.MiddleCenter, 1040, 90);
        _codexProgressText.color = Violet;
        _codexPageObjects.Add(_codexProgressText.gameObject);
        for (int i = 0; i &lt; 10; i++)
        {
            int col = i % 5, row = i / 5;
            var slot = MakeText(_panel, "G" + (i + 1), 40, new Vector2(-400 + col * 200, -30 - row * 150),
                TextAnchor.MiddleCenter, 190, 120);
            slot.color = new Color(0.35f, 0.30f, 0.45f);
            _codexSlotTexts.Add(slot);
            _codexPageObjects.Add(slot.gameObject);
        }
        _codexBeltText = MakeText(_panel, "", 36, new Vector2(0, -340), TextAnchor.MiddleCenter, 1040, 80);
        _codexBeltText.color = new Color(0.85f, 0.80f, 0.95f);
        _codexPageObjects.Add(_codexBeltText.gameObject);
        _codexPageObjects.Add(MakeText(_panel, UiText.Get("talent_codex_foot"), 34,
            new Vector2(0, -430), TextAnchor.MiddleCenter, 1040, 70).gameObject);
        _codexPageObjects.Add(MakeButtonOn(_panel, UiText.Get("talent_to_talent"), new Vector2(-430, 620), new Vecto
            new Color(0.30f, 0.42f, 0.55f, 0.95f), () =&gt; TalentPageRequested?.Invoke()).gameObject);
        MakeButtonOn(_panel, UiText.Get("talent_close"), new Vector2(0, -560), new Vector2(420, 100),
            new Color(0.16f, 0.18f, 0.24f, 0.9f), () =&gt; CloseRequested?.Invoke());
        _hintText = MakeText(_panel, "", 34, new Vector2(0, -690), TextAnchor.MiddleCenter, 1040, 90);
        _hintText.color = new Color(0.95f, 0.87f, 0.6f);
        _panel.gameObject.SetActive(false);
    }
    /// &lt;summary&gt;Flip the panel between the talent page and the codex page
    /// (shared chrome: title/day/wallet/close/hint stay put).&lt;/summary&gt;
    void SetPage(bool codex)
    {
        for (int i = 0; i &lt; _talentPageObjects.Count; i++) _talentPageObjects[i].SetActive(!codex);
        for (int i = 0; i &lt; _codexPageObjects.Count; i++) _codexPageObjects[i].SetActive(codex);
        _titleText.text = codex ? UiText.Get("talent_codex_title") : UiText.Get("talent_title");
    }
    /// &lt;summary&gt;Open/refresh the panel: branch rows rebuilt from the face data
    /// (points/next cost/milestone move every buy, a cached layout would lie),
    /// codex slots light per the claim latch, chain line shows today's task.&lt;/summary&gt;
    public void ShowTalent(int dayIndex, int walletCoins, IReadOnlyList&lt;BranchFace&gt; branches,
                           int nextPointCost, string chainStatus, bool[] weekClaimed)
    {
        SetPage(codex: false);
```

```
        _dayLabel.text = UiText.Get("talent_day_fmt", dayIndex.ToString());
        SetWallet(walletCoins);
        _chainText.text = chainStatus;
        SetHint(UiText.Get("talent_hint_talent"));
        _branchIds.Clear();
        int rows = Math.Min(3, branches != null ? branches.Count : 0);
        for (int i = 0; i &lt; rows; i++)
        {
            var face = branches[i];
            _branchIds.Add(face.Id);
            var row = _panel.Find("BranchRow" + i);
            var texts = row.GetComponentsInChildren&lt;Text&gt;();
            // texts[0]=main, texts[1]=desc, texts[2]=buy label (factory order).
            texts[0].text = UiText.Get("talent_branch_row_fmt", face.Name, face.Points.ToString(),
                face.Max.ToString(), nextPointCost.ToString());
            texts[1].text = face.NextMilestoneDesc;
        }
        for (int i = 0; i &lt; _slotTexts.Count &amp;&amp; i &lt; weekClaimed.Length; i++)
        {
            bool lit = weekClaimed[i];
            _slotTexts[i].text = lit ? "D" + (i + 1) + " " : "D" + (i + 1);
            _slotTexts[i].color = lit ? NeonCyan : new Color(0.35f, 0.30f, 0.45f);
        }
        _panel.gameObject.SetActive(true);
    }
    /// &lt;summary&gt;One codex roster slot's display face (the launcher builds it from
    /// the cfg_enemy roster order + the session collection; names carry the 05 s4
    /// first-version authority until the loc table lands with the art batch).&lt;/summary&gt;
    public struct CodexSlotFace
    {
        public string CodeName;
        public string NameCn;
        public bool Collected;
    }
    /// &lt;summary&gt;Open/refresh the codex page: roster slots light per the collection,
    /// the progress line reads the live bonus (CollectionBook math, honest data),
    /// the belt line shows the week-chain task.&lt;/summary&gt;
    public void ShowCodex(int dayIndex, int walletCoins, IReadOnlyList&lt;CodexSlotFace&gt; roster,
                          int collected, int total, float globalPct, string chainStatus)
    {
        SetPage(codex: true);
        _dayLabel.text = UiText.Get("talent_day_fmt", dayIndex.ToString());
        SetWallet(walletCoins);
        _codexProgressText.text = UiText.Get("talent_codex_progress_fmt", collected.ToString(),
            total.ToString(), (globalPct * 100f).ToString("0.#"));
        _codexBeltText.text = chainStatus;
        SetHint(UiText.Get("talent_hint_codex"));
        int n = Math.Min(_codexSlotTexts.Count, roster != null ? roster.Count : 0);
        for (int i = 0; i &lt; n; i++)
        {
```

```
            var f = roster[i];
            _codexSlotTexts[i].text = f.CodeName + " " + f.NameCn + (f.Collected ? "  " : "");
            _codexSlotTexts[i].color = f.Collected ? NeonCyan : new Color(0.35f, 0.30f, 0.45f);
        }
        _panel.gameObject.SetActive(true);
    }
    /// &lt;summary&gt;Buy success beat: violet flash with the branch name.&lt;/summary&gt;
    public void PlayBuyFlash(string branchName)
    {
        StartCoroutine(BuyFlashRoutine(branchName));
    }
    IEnumerator BuyFlashRoutine(string branchName)
    {
        var overlay = MakeOverlayPanel("BuyFlash", new Color(0.06f, 0.04f, 0.10f, 0.9f));
        var main = MakeText(overlay, UiText.Get("talent_buy_flash_fmt", branchName), 80, new Vector2(0, 90),
            TextAnchor.MiddleCenter, 1040, 130);
        main.color = Violet;
        PlayTone(587f, 0.16f);
        yield return new WaitForSeconds(0.7f);
        Destroy(overlay.gameObject);
    }
    public void SetWallet(int coins) =&gt; _walletText.text = UiText.Get("talent_wallet_fmt", coins.ToString());
    public void SetHint(string hint) =&gt; _hintText.text = hint;
    public void Hide() =&gt; _panel.gameObject.SetActive(false);
    public bool IsVisible =&gt; _panel != null &amp;&amp; _panel.gameObject.activeSelf;
    // ——— factory helpers (DayView proven pattern) ———
    Button MakeButtonOn(Transform parent, string text, Vector2 pos, Vector2 size, Color color, Action onClick)
    {
        var go = new GameObject(text, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = size;
        var image = go.GetComponent&lt;Image&gt;();
        image.sprite = _square;
        image.color = color;
        MakeText(go.transform, text, 40, Vector2.zero, TextAnchor.MiddleCenter, size.x, size.y).color = BoneWhite;
        var button = go.GetComponent&lt;Button&gt;();
        button.onClick.AddListener(() =&gt; onClick());
        return button;
    }
    RectTransform MakeOverlayPanel(string name, Color color)
    {
        var go = new GameObject(name, typeof(RectTransform), typeof(Image));
        go.transform.SetParent(_canvas.transform, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = Vector2.zero;
        rt.anchorMax = Vector2.one;
        rt.offsetMin = rt.offsetMax = Vector2.zero;
```

```
            var image = go.GetComponent&lt;Image&gt;();
            image.sprite = _square;
            image.color = color;
            return rt;
        }
        static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h)
        {
            var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
            go.transform.SetParent(parent, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = pos;
            rt.sizeDelta = new Vector2(w, h);
            var t = go.GetComponent&lt;Text&gt;();
            t.font = UiFont.Resolve();               // T035: FontSet-driven CJK chain
            t.text = text;
            t.fontSize = size;
            t.alignment = anchor;
            t.color = new Color(0.91f, 0.89f, 0.85f);
            t.supportRichText = true;
            return t;
        }
        static Sprite MakeSquareSprite()
        {
            var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
            var px = tex.GetPixels32();
            for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
            tex.SetPixels32(px);
            tex.Apply(false, false);
            return Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
        }
        // Procedural placeholder tone (P08 pattern; formal clips swap at M2.5).
        void PlayTone(float freq, float seconds)
        {
            if (_toneSource == null) _toneSource = gameObject.AddComponent&lt;AudioSource&gt;();
            const int sr = 22050;
            int n = Mathf.Max(1, (int)(sr * seconds));
            var data = new float[n];
            for (int i = 0; i &lt; n; i++)
            {
                float k = 1f - (float)i / n;
                data[i] = Mathf.Sin(2f * Mathf.PI * freq * i / sr) * k;
            }
            var clip = AudioClip.Create("tone", n, 1, sr, false);
            clip.SetData(data, 0);
            _toneSource.PlayOneShot(clip);
        }
    }
}
// TutorialFlow.cs
```

```
using System;
namespace Sy.P01.Newbie
{
    /// &lt;summary&gt;
    ///        evt_newbie_step 1-5 ①   ②     ③    ④   Boss⑤   +
    ///      BattleSim   /W3     /      Boss           /  /    ——
    /// Play   headless   NewbieFlowCheck
    /// &lt;/summary&gt;
    public sealed class TutorialFlow
    {
        public int Step { get; private set; }
        readonly Action&lt;string&gt; _hint;
        readonly Action&lt;int&gt; _onStep;
        public TutorialFlow(BattleSim sim, Action&lt;string&gt; hint, Action&lt;int&gt; onStep)
        {
            _hint = hint ?? delegate { };
            _onStep = onStep ?? delegate { };
            sim.FirstMoved += OnFirstMoved;
            sim.TalismanOffered += OnTalismanOffered;
            sim.TalismanPicked += OnTalismanPicked;
            sim.EliteKilled += OnEliteKilled;
            sim.BossKilled += OnBossKilled;
            sim.StateChanged += OnStateChanged;
            _hint(UiText.Get("tutorial_open"));
        }
        void Advance(int step)
        {
            if (Step &gt;= step) return;
            Step = step;
            _onStep(step);
        }
        void OnFirstMoved()
        {
            Advance(1);
            _hint(UiText.Get("tutorial_moved"));
        }
        void OnTalismanOffered()
        {
            _hint(UiText.Get("tutorial_offer_fmt", NewbieConfig.EvolutionName));
        }
        void OnTalismanPicked(int index)
        {
            Advance(2);
            var t = NewbieConfig.Talismans[index];
            _hint(UiText.Get("tutorial_pick_fmt", t.Name, NewbieConfig.ChainPending(NewbieConfig.TalismanChainChars[inde
        }
        void OnEliteKilled()
        {
            Advance(3);
            _hint(UiText.Get("tutorial_elite_fmt", NewbieConfig.ChainComplete(NewbieConfig.TalismanChainChars[0])));
```

```
        }
        void OnBossKilled()
        {
            Advance(4);
            _hint(UiText.Get("tutorial_boss"));
        }
        void OnStateChanged(BattleSim.BattleState state)
        {
            if (state == BattleSim.BattleState.Won)
            {
                Advance(5);
                _hint(UiText.Get("tutorial_won"));
            }
        }
    }
}
// WxBoardPostChannel.cs
#if SY_WECHAT
// X129 law: the DllImport surface only exists on armed weixin builds
// (BuildWeixin injects SY_WECHAT) - Editor/mock/douyin builds never compile
// this file, so the armed command pump and its jslib import simply do not
// exist there (the service entry is the honest no-op, zero drift).
using System.Runtime.InteropServices;
namespace Sy.P01.Newbie
{
    /// &lt;summary&gt;
    /// T036b: the armed main-domain board command pump. wx.getOpenDataContext
    /// ().postMessage is the ONLY channel into the open-data subdomain: every
    /// board open posts one idempotent refresh command, the subdomain re-reads
    /// the friend KV (getFriendCloudStorage) and repaints the sharedCanvas.
    /// The payload rides the shared jslib shape ({boardId, viewport:payload})
    /// and carries the session's own kills for the always-present "You"
    /// footer (the subdomain has no self-id API - the self row can never be
    /// flagged by matching, only stated by the main-domain truth). Fire-and-
    /// forget: the post is the honest unit; the painted board is the U014
    /// physical face. Zero shared-package changes (X236 law - same symbol
    /// the com.sy.platform bridge declares, same shape as the write channel).
    /// &lt;/summary&gt;
    public static class WxBoardPoster
    {
        public const string BoardId = "friend_kills";
        public static void PostRefresh(int selfKills)
            =&gt; WxBoardPostBridge.SyOpenDataPost(
                BoardId, "{\"selfKills\":" + selfKills + "}");
    }
    /// &lt;summary&gt;P01-local import of the shared jslib export (the jslib ships
    /// with the shared platform package this project references - zero
    /// shared-package changes, X236 law; distinct bridge name so the write
    /// channel's bridge and this one never collide).&lt;/summary&gt;
    static class WxBoardPostBridge
```

```
    {
        [DllImport("__Internal")] internal static extern void SyOpenDataPost(
            string boardPtr, string payloadPtr);
    }
}
#endif
// WxBoardRelayChannel.cs
#if SY_WECHAT
// X129 law: the DllImport surface only exists on armed weixin builds
// (BuildWeixin injects SY_WECHAT) - Editor/mock/douyin builds never compile
// this file, so the armed relay sink and its jslib import simply do not
// exist there (the relay default keeps the no-op, zero drift).
using System.Runtime.InteropServices;
namespace Sy.P01.Newbie
{
    /// &lt;summary&gt;
    /// T036c: the armed WX board relay sink. The P01 wx package is a
    /// self-built thin adapter (no official unity-sdk JS), so the display
    /// chain rides the official ScreenCanvas mode directly through the
    /// P01-local SyBoardRelay jslib: wx.getOpenDataContext({sharedCanvasMode:
    /// 'screenCanvas'}) locks the shared singleton as an on-screen overlay
    /// (base library 3.6.6+; older clients ignore the option and keep the
    /// native delegation ladder - never empty on any device). RelayShow
    /// positions the overlay over the board row strip (device px, the
    /// relay service's rect math); RelayHide applies the official 1px
    /// off-screen hide recipe. The launcher posts one refresh after every
    /// mount (PostBoardRefresh): the main-domain resize clears the canvas
    /// by canvas law, the subdomain repaints it. Zero shared-package
    /// changes (X236 law - P01-local jslib, same bridge shape as the
    /// write/post channels).
    /// &lt;/summary&gt;
    public sealed class WxBoardRelaySink : IBoardRelaySink
    {
        public void RelayShow(int x, int y, int w, int h)
            =&gt; WxBoardRelayBridge.SyBoardRelayShow(x, y, w, h);
        public void RelayHide()
            =&gt; WxBoardRelayBridge.SyBoardRelayHide();
    }
    /// &lt;summary&gt;P01-local import of the P01-local jslib export (the jslib
    /// ships in Assets/_Game/Plugins - zero shared-package changes, X236
    /// law; distinct symbol family from the shared bridge exports).&lt;/summary&gt;
    static class WxBoardRelayBridge
    {
        [DllImport("__Internal")] internal static extern void SyBoardRelayShow(
            int x, int y, int w, int h);
        [DllImport("__Internal")] internal static extern void SyBoardRelayHide();
    }
}
#endif
// WxBoardWriteChannel.cs
```

```
#if SY_WECHAT
// X129 law: the DllImport surface only exists on armed weixin builds
// (BuildWeixin injects SY_WECHAT) - Editor/mock/douyin builds never compile
// this file, so the armed writer face and its jslib import simply do not
// exist there (the channel default keeps the honest no-op, zero drift).
using System.Runtime.InteropServices;
namespace Sy.P01.Newbie
{
    /// &lt;summary&gt;
    /// T036a: the armed main-domain write face. wx.setUserCloudStorage is
    /// the MAIN-domain KV write (the only domain allowed to write the self
    /// score; friend rows stay subdomain-read-only until T036b). The shared
    /// jslib export is fire-and-forget (the T013-B2 symbol ships no result
    /// callback), so the dispatch is the honest unit - the real &amp;lt;=1s
    /// write clock is the U014 physical face. Exactly-once per value change:
    /// the BoardWriteOnceGuard swallows duplicate pushes, so a board
    /// re-open with an unchanged score never re-writes (KV field lock: the
    /// payload only ever goes out under OpenDataBoardService.KvKeyKills).
    /// &lt;/summary&gt;
    public sealed class WxBoardWriteFace : IOpenDataBoardWriter
    {
        static readonly BoardWriteOnceGuard Guard = new();
        public bool WriteSelfKv(int kills)
            =&gt; Guard.TryPush(kills, v =&gt;
                WxBoardBridge.SySetUserCloudStorage(
                    OpenDataBoardService.KvKeyKills, v.ToString()));
    }
    /// &lt;summary&gt;P01-local import of the shared jslib export (same symbol
    /// the com.sy.platform bridge declares; the jslib ships with the shared
    /// package this project references - zero shared-package changes, X236
    /// law, same shape as WxSubscribeBridge).&lt;/summary&gt;
    static class WxBoardBridge
    {
        [DllImport("__Internal")] internal static extern void SySetUserCloudStorage(
            string kvKey, string kvValue);
    }
}
#endif
// WxOpenDataBoard.cs
using System.Collections.Generic;
namespace Sy.P01.Newbie
{
    /// &lt;summary&gt;
    /// T022-iv: the armed-build friend kills board face. Real WX friend
    /// scores only exist inside the open data domain - the subdomain package
    /// + sharedCanvas feed is T036's row - so this face is the armed SOCKET:
    /// until that feed lands it serves the exact pseudo-friend ladder the
    /// mock serves, by DELEGATION (one roster source, no copied data, zero
    /// drift by construction; the board never dead-ends and never fabricates
    /// telemetry). When T036 wires the real feed in, this class grows the
```

```
    /// real-row source and the mock stays the Editor/gate face.
    /// Pure managed delegation - no DllImport/jslib native surface - so it
    /// compiles on every platform (unlike WxSubscribeFace, X129 native law);
    /// the DefaultBoardFaceFactory's SY_WECHAT define gates WHO uses it on
    /// each build, and FriendBoardCheck F8 asserts the delegation identity
    /// directly in plain Editor gates.
    /// Real-device verification ("friend board visible on a real phone") is
    /// the user physical face (U014) - this code face is N/A-justified there.
    /// &lt;/summary&gt;
    public sealed class WxOpenDataBoard : IOpenDataBoard
    {
        public List&lt;FriendBoardRow&gt; GetFriendKillsBoard(int selfKills, int maxRows)
            =&gt; new MockOpenDataBoard().GetFriendKillsBoard(selfKills, maxRows);
    }
}
// WxSubscribeFace.cs
#if SY_WECHAT
// X129 law: the DllImport surface only exists on armed weixin builds
// (BuildWeixin injects SY_WECHAT) - Editor/mock/douyin builds never compile
// this file, so the default face stays the local mock (SubscribeService).
namespace Sy.P01.Newbie
{
    /// &lt;summary&gt;
    /// T022-iv-B: the armed-build WX ask face over the P01-local jslib result
    /// channel (WxSubscribeJsChannel). Placeholder law (G5/T001c): template
    /// ids carrying the placeholder prefix are NOT real console ids and never
    /// reach the device - the ask fails honestly as NotAsked (no ledger burn,
    /// no funnel events). Real console ids fire wx.requestSubscribeMessage
    /// through the channel; the sync return stays the honest pending Denied -
    /// the verified device grant (T022-iv-B result channel) is the only thing
    /// that ever lands a grant, never a fabricated sync grant.
    /// &lt;/summary&gt;
    public sealed class WxSubscribeFace : ISubscribeFace
    {
        public SubscribeOutcome Request(P01Configs.SubscribeRow row, System.Action&lt;bool&gt; onDeviceResult)
        {
            string tmplId = row.wxTemplateId;
            if (string.IsNullOrEmpty(tmplId)
                || tmplId.StartsWith(SubscribeService.PlaceholderPrefix))
                return SubscribeOutcome.NotAsked;
            // Real console id: fire the one-shot subscribe dialog. The
            // service records the ask (ledger + request event) on the sync
            // path; the granted stage only ever fires when the device
            // result arrives verified through the channel callback.
            WxSubscribeJsChannel.Ask("[\"" + tmplId + "\"]", onDeviceResult);
            return SubscribeOutcome.Denied;
        }
    }
}
#endif
```

```
// WxSubscribeJsChannel.cs
#if SY_WECHAT
// X129 law: the DllImport surface only exists on armed weixin builds
// (BuildWeixin injects SY_WECHAT) - Editor/mock/douyin builds never compile
// this file, so the channel and its scene relay simply do not exist there.
using System;
using System.Collections.Generic;
using System.Runtime.InteropServices;
using UnityEngine;
namespace Sy.P01.Newbie
{
    /// &lt;summary&gt;
    /// T022-iv-B: the P01-local jslib result channel for subscribe asks.
    /// The shared com.sy.platform jslib resolves the wasm instance and calls
    /// SendMessage('SyJsBridge', 'OnJsResult', cbId + '\u001F' + json) - a
    /// broadcast every MonoBehaviour on the host object receives. This channel
    /// owns the P01 half of that contract with ZERO shared-package changes
    /// (P02/P06 pending-review byte stability, X236 law):
    ///  - the host object is the same 'SyJsBridge' the shared suite uses;
    ///  - the shared public JsBridgeReceiver component keeps serving suite
    ///    callbacks (ads/login/album) no matter which side built the host;
    ///  - the local relay component feeds P01 subscribe results only.
    /// Creation order is order-proof: whichever side builds the host first,
    /// the other finds it and only adds its missing component, so a P01
    /// subscribe ask can never orphan a suite callback and vice versa.
    /// &lt;/summary&gt;
    static class WxSubscribeJsChannel
    {
        static readonly Dictionary&lt;string, Action&lt;bool&gt;&gt; Pending = new();
        static int _nextId;
        static bool _hostReady;
        /// &lt;summary&gt;Fire the ask and register the result callback. The jslib
        /// answers exactly once ({ok:true/false}) through the wx success/fail
        /// pair, delivered on the main thread by SendMessage - safe to touch
        /// the save slot directly from the callback.&lt;/summary&gt;
        internal static void Ask(string tmplIdsJson, Action&lt;bool&gt; onDeviceResult)
        {
            if (onDeviceResult == null) return;   // no listener: nothing to wire
            EnsureHost();
            string cbId = "p01_subscribe_" + ++_nextId;
            Pending[cbId] = onDeviceResult;
            WxSubscribeBridge.SyRequestSubscribe(tmplIdsJson, cbId);
        }
        /// &lt;summary&gt;Route one jslib result. Unknown ids (the shared suite's
        /// own callbacks) are ignored untouched; a malformed payload is an
        /// honest deny.&lt;/summary&gt;
        internal static void Complete(string id, string json)
        {
            if (id == null) return;
            if (!Pending.Remove(id, out var onResult)) return;
```

```
            bool ok = false;
            try
            {
                var data = Sy.Core.MiniJson.Deserialize(json) as Dictionary&lt;string, object&gt;;
                if (data != null &amp;&amp; data.TryGetValue("ok", out var v)) ok = Convert.ToBoolean(v);
            }
            catch (Exception) { ok = false; }
            onResult(ok);
        }
        static void EnsureHost()
        {
            if (_hostReady) return;
            var host = GameObject.Find("SyJsBridge");
            if (host == null)
            {
                host = new GameObject("SyJsBridge");
                UnityEngine.Object.DontDestroyOnLoad(host);
            }
            if (host.GetComponent&lt;Sy.Platform.Wx.JsBridgeReceiver&gt;() == null)
                host.AddComponent&lt;Sy.Platform.Wx.JsBridgeReceiver&gt;();
            if (host.GetComponent&lt;WxSubscribeResultRelay&gt;() == null)
                host.AddComponent&lt;WxSubscribeResultRelay&gt;();
            _hostReady = true;
        }
    }
    /// &lt;summary&gt;Scene relay on the shared host: SendMessage from the jslib
    /// broadcasts OnJsResult to every component - this one routes P01
    /// subscribe results while the shared receiver keeps its own registry
    /// (both components always coexist on the single host object).&lt;/summary&gt;
    [DisallowMultipleComponent]
    sealed class WxSubscribeResultRelay : MonoBehaviour
    {
        public void OnJsResult(string message)
        {
            int split = message.IndexOf('\u001F');
            string cbId = split &lt; 0 ? message : message.Substring(0, split);
            string json = split &lt; 0 ? "" : message.Substring(split + 1);
            WxSubscribeJsChannel.Complete(cbId, json);
        }
    }
    /// &lt;summary&gt;P01-local import of the shared jslib export (same symbol as
    /// Sy.Platform.Wx.WxBridge.SyRequestSubscribe; the jslib ships with the
    /// com.sy.platform package this project references).&lt;/summary&gt;
    static class WxSubscribeBridge
    {
        [DllImport("__Internal")] internal static extern void SyRequestSubscribe(
            string tmplIdsJson, string cbId);
    }
}
#endif
```